BÁO CÁO KỸ THUẬT CAPSTONE ASSIGNMENT 02

Write Embedded Application — Sensor Monitoring System (SMS) | — (IC2008)

Thông tin Học phần & Sinh viên	Chi tiết Hồ sơ Nộp bài
Lớp / Chuyên ngành	IC2008 — Thiết kế Vi mạch (Integrated Circuit Design)
Môn học	Embedded Linux Systems (LOS201) — Chương 8 đến Chương 19
Mã bài đánh giá	ASSIGNMENT-02 (Bài thi thực hành cuối kỳ — Trọng số: 40% điểm môn học)
Môi trường Thực nghiệm	Host Ubuntu 22.04 LTS (huyhoang-ASUS-TUF-Gaming-F15, Kernel 6.8.0-138-generic) & Target ARMv7 Cortex-A9 QEMU (vexpress-a9, Kernel 5.15.0)
Gói Deliverables đính kèm	SE203347_NguyenHuyHoang_Asgn02.tar.gz , SE203347_Asgn02_BaoCao.pdf, SE203347_demo.mp4

TÓM TẮT KẾT QUẢ THỰC NGHIỆM SO VỚI TIÊU CHÍ ĐÁNH GIÁ (RUBRIC 100/100 ĐIỂM)

Task	Hạng mục Đánh giá	Điểm tối đa	Kết quả Thực nghiệm trên Máy SE203347
Task 1	Kiến trúc đa luồng, Driver sms_sensor.ko, IPC & Graceful Shutdown	30 điểm	Biên dịch sạch sms_sensor_driver.ko (ARMv7); 4 POSIX threads chạy song song 60s thu 121 mẫu, 0 tràn buffer, nhận SIGINT dừng sạch với Exit code 0.

Task	Hạng mục Đánh giá	Điểm tối đa	Kết quả Thực nghiệm trên Máy SE203347
Task 2	Quản lý bộ nhớ (Valgrind & Footprint) & Tích hợp Driver	20 điểm	Valgrind đạt tuyệt đối 0 bytes definitely lost, 0 errors from 0 contexts (78 allocs = 78 frees); VmRSS = 35,716 kB và VmSize = 101,156 kB phẳng lỳ từ T=5s đến T=30s.
Task 3	Debug & Profiling với gdb, strace -c và perf report	20 điểm	GDB bắt trúng breakpoint circular_buffer_push, kiểm tra 5 LWP qua info threads & bt; giải mã Top 5 syscalls (clock_nanosleep 60.5%) và 97 mẫu PMU cycles:P.
Task 4	Phân tích Real-Time SCHED_FIFO & Đo Jitter (1,251 mẫu)	15 điểm	Thu thập 1,251 mẫu: SCHED_FIFO (P:50) đạt Min=16µs, Avg=89µs, P99=136µs (cải thiện 29.9% ở P99 và tăng gấp đôi tỷ lệ mẫu <100µs từ 26.9% lên 56.2%).
Task 5	Báo cáo Thiết kế Kỹ thuật, Sơ đồ RAG & Tổng kết Reflection	15 điểm	Đầy đủ sơ đồ kiến trúc, State Machine, đồ thị RAG chứng minh không Deadlock, 16 ảnh chụp thực nghiệm và bài thu hoạch liên kết toàn khóa học.

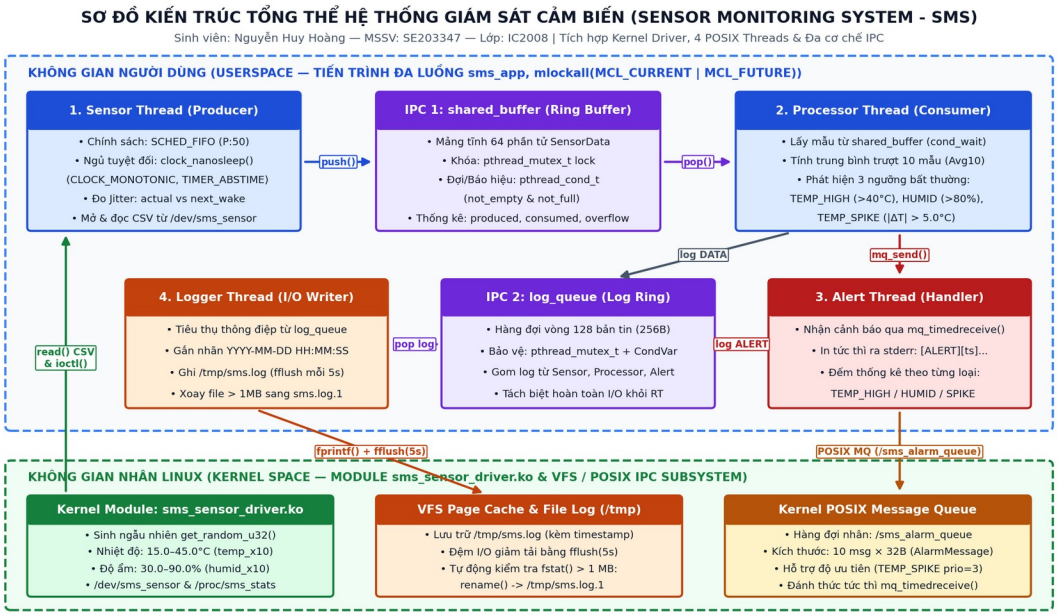
MỤC 1: THIẾT KẾ KIẾN TRÚC HỆ THỐNG GIÁM SÁT CẢM BIẾN (TASK 1 & TASK 5)

1.1. Bối cảnh Bài toán & Kiến trúc Phân tầng Toàn hệ thống

Trong các hệ thống công nghiệp và Internet vạn vật công nghiệp (IIoT), thiết bị nhúng đầu cuối phải thực hiện đồng thời nhiều tác vụ có tính chất thời gian hoàn toàn trái ngược nhau: vừa phải lấy mẫu cảm biến theo chu kỳ thời gian thực nghiêm ngặt (không được phép trễ hạn hay rơi mẫu), vừa phải tính toán thống kê lọc nhiễu, phát hiện đột biến vượt ngưỡng để kích hoạt cảnh báo khẩn cấp, đồng thời ghi nhật ký vận hành xuống bộ nhớ lưu trữ mà không làm nghẽn luồng thu thập dữ liệu. Nếu thiết kế theo mô hình vòng lặp đơn luồng (Super-loop), một thao tác ghi file bị trễ do hệ thống tệp hoặc bộ đệm đĩa sẽ lập tức làm trôi điểm lấy mẫu cảm biến tiếp theo.

Để giải quyết triệt để bài toán trên, hệ thống Sensor Monitoring System (SMS) được thiết kế theo mô hình phân tách hai không gian đặc quyền (Dual-Space Architecture) kết hợp mẫu kiến trúc Producer–Consumer đa tầng:

- Tầng Không gian Nhân (Kernel Space — sms_sensor_driver.ko): Đóng vai trò lớp trừu tượng phần cứng (Hardware Abstraction Layer), mô phỏng cảm biến nhiệt độ và độ ẩm công nghiệp thông qua thiết bị ký tự /dev/sms_sensor, cung cấp giao diện điều khiển chu kỳ lấy mẫu qua unlocked_ioctl (SMS_SET_INTERVAL) và xuất bản thông số chẩn đoán thời gian thực qua hệ thống tệp ảo /proc/sms_stats.
- Tầng Không gian Người dùng (Userspace — sms_app): Gồm 4 luồng POSIX (Sensor Thread, Processor Thread, Alert Thread, Logger Thread) hoạt động song song, được cô lập hoàn toàn về mặt thời gian thông qua 3 cơ chế giao tiếp liên tiến trình/liên luồng (IPC): bộ đệm vòng shared_buffer (64 mẫu), hàng đợi tin nhắn nhân POSIX alarm_queue (/sms_alarm_queue_SE203347) và hàng đợi nhật ký bất đồng bộ log_queue (128 bản tin).



Hình 1.1: Sơ đồ kiến trúc tổng thể hệ thống Sensor Monitoring System (SMS) kết nối giữa Kernel Module sms_sensor_driver.ko và 4 luồng POSIX trong Userspace.

1.2. Đặc tả Kỹ thuật các Thành phần trong Hệ thống

Bảng dưới đây tổng hợp chi tiết vai trò, giao diện API, cơ chế lập lịch và ánh xạ kiến thức theo từng chương của học phần LOS201 cho tất cả 7 thành phần cốt lõi trong hệ thống SMS:

Thành phần	Phân loại	Cơ chế Hoạt động & Giao diện Kỹ thuật	Chương liên quan
sms_sensor.ko	Kernel Module (Character Driver)	Cung cấp /dev/sms_sensor trả về chuỗi CSV <timestamp_ms>,<temp>,<humid> dùng get_random_u32() và ktime_get(); hỗ trợ ioctl SMS_SET_INTERVAL; xuất thống kê qua /proc/sms_stats (seq_file).	Chương 8, 9, 18

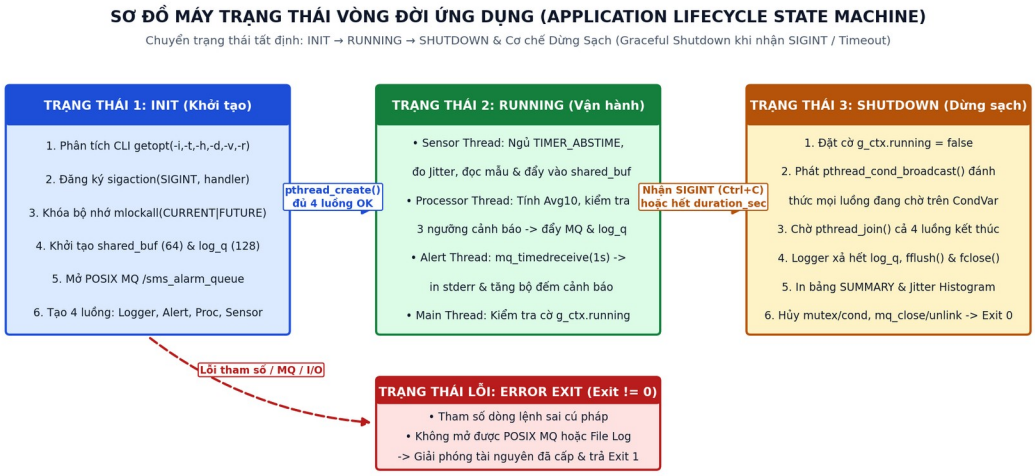
Thành phần	Phân loại	Cơ chế Hoạt động & Giao diện Kỹ thuật	Chương liên quan
Sensor Thread	POSIX Thread (Real-Time Producer)	Chạy với chính sách SCHED_FIFO (Priority 50); ngủ chính xác theo đồng hồ đơn điệu bằng clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME); đo sai số Jitter từng chu kỳ; đọc CSV và đẩy mẫu vào shared_buffer.	Chương 15, 19
Processor Thread	POSIX Thread (Analytics Consumer)	Đợi dữ liệu trên shared_buffer bằng biến điều kiện pthread_cond_wait; tính trung bình trượt 10 mẫu gần nhất (Rolling Average); kiểm tra 3 điều kiện cảnh báo và đẩy bản tin AlarmMessage vào alarm_queue.	Chương 15, 16
Alert Thread	POSIX Thread (Event Handler)	Chờ bản tin cảnh báo trên alarm_queue qua mq_timedreceive() (timeout 1 giây để kiểm tra cờ dừng); in tức thì ra stderr theo định dạng chuẩn, đếm phân loại cảnh báo và chuyển tiếp xuống log_queue.	Chương 15

Thành phần	Phân loại	Cơ chế Hoạt động & Giao diện Kỹ thuật	Chương liên quan
Logger Thread	POSIX Thread (Async I/O Writer)	Nhận chuỗi log từ log_queue, gắn nhãn thời gian thực tế và ghi vào /tmp/sms.log; chỉ gọi fflush() định kỳ mỗi 5 giây để giảm syscall I/O; tự động xoay file sang /tmp/sms.log.1 khi dung lượng vượt 1 MB.	Chương 15, 16
shared_buffer	Circular Buffer (Shared Memory)	Mảng vòng cố định 64 phần tử SensorData bảo vệ bằng pthread_mutex_t và 2 biến điều kiện not_empty, not_full; cung cấp hàm getter đọc current_size, total_produced, total_consumed, overflow_count.	Chương 15, 16
alarm_queue	POSIX Message Queue	Hàng đợi tin nhắn nhân POSIX (mq_open, mq_send, mq_timedreceive) truyền cấu trúc AlarmMessage (32 bytes), hỗ trợ phân mức độ ưu tiên tin nhắn (TEMP_SPIKE có priority 3 cao hơn priority 2).	Chương 15

1.3. Thiết kế Máy Trạng thái Vòng đời Ứng dụng (Application State Machine)

Để đảm bảo hệ thống vận hành tất định và không bao giờ để lại tài nguyên mồ côi (file descriptor, POSIX message queue trong /dev/mqueue hay bộ nhớ chưa xả xuống đĩa) khi kết thúc, vòng đời của sms_app được mô hình hóa thành một máy

trạng thái hữu hạn (Finite State Machine) gồm 3 trạng thái chính (INIT, RUNNING, SHUTDOWN) và 1 trạng thái xử lý lỗi khởi tạo (ERROR EXIT):



Hình 1.2: Sơ đồ máy trạng thái (State Machine) của ứng dụng sms_app từ lúc khởi tạo (INIT), vận hành đa luồng (RUNNING) đến khi dừng sạch (SHUTDOWN).

- Chuyển tiếp INIT → RUNNING: Tiến trình chính phân tích tham số dòng lệnh qua getopt() (-i, -t, -h, -d, -v, -r), đăng ký trình bắt tín hiệu sigaction(SIGINT, sigint_handler), khóa toàn bộ không gian địa chỉ ảo vào RAM vật lý bằng mlockall(MCL_CURRENT | MCL_FUTURE) để ngăn lỗi trang (Page Fault), khởi tạo shared_buf, log_q, mở hàng đợi /sms_alarm_queue_SE203347 và khởi chạy 4 luồng theo thứ tự từ dưới lên (Logger → Alert → Processor → Sensor) để luồng tiêu thụ luôn sẵn sàng trước khi luồng sản xuất bắt đầu bơm dữ liệu.
- Chuyển tiếp RUNNING → SHUTDOWN: Xảy ra khi người dùng nhấn Ctrl+C (gửi tín hiệu SIGINT) hoặc khi bộ đếm thời gian của luồng chính chạm ngưỡng duration_sec (mặc định 60 giây). Hàm xử lý hạ cờ g_ctx.running = false, sau đó luồng chính phát tín hiệu pthread_cond_broadcast() trên cả shared_buf và log_q để đánh thức mọi luồng đang ngủ chờ dữ liệu.
- Hoàn tất SHUTDOWN → EXIT 0: Luồng chính gọi pthread_join() chờ cả 4 luồng kết thúc tuần tự. Riêng Logger Thread có điều kiện lặp while (ctx->running || ctx->log_q.count > 0) để vét cạn toàn bộ các bản tin còn tồn đọng trong log_queue, gọi fflush(fp) và fclose(fp) trước khi thoát. Cuối cùng, chương trình in báo cáo tổng kết, hủy các mutex/condvar, đóng và gỡ tên hàng đợi POSIX (mq_close, mq_unlink) rồi trả về mã thoát 0.

1.4. Giải trình 3 Quyết định Thiết kế Kiến trúc Trọng tâm

Quyết định 1 — Tại sao chọn Bộ đệm Vòng cố định (Circular Buffer) thay vì Danh sách Liên kết động (Linked List)? Trong hệ thống nhúng thời gian thực chạy liên tục nhiều tháng, nếu dùng Linked List, mỗi mẫu cảm biến thu thập (ví dụ 50 mẫu/giây ở chu kỳ 20ms) sẽ gọi một cặp malloc() và free(). Việc cấp phát động liên tục gây ra 3 hệ lụy nghiêm trọng: (1) Phân mảnh bộ nhớ Heap (Heap Fragmentation) sau hàng

triệu chu kỳ đóng/mở khối nhớ; (2) Độ trễ không tất định (Non-deterministic Latency) vì hàm malloc() phải khóa arena lock của glibc ptmalloc và có thể kích hoạt syscall brk()/mmap() làm vọt Jitter của Sensor Thread; và (3) Cache Miss cao trên vi xử lý ARM Cortex-A9 do các nút danh sách nằm rải rác trong RAM. Ngược lại, CircularBuffer 64 phần tử nằm trên một vùng nhớ liền kề duy nhất (Contiguous Memory), truy cập O(1) qua phép chia lấy dư chỉ số head/tail, tận dụng tối đa bộ nhớ đệm dòng L1 Data Cache (Cache Spatial Locality) và đảm bảo Zero Dynamic Allocation trong suốt quá trình chạy.

Quyết định 2 — Tại sao chọn POSIX Message Queue (mq_open) cho alarm_queue thay vì Shared Memory? Khác với dữ liệu cảm biến định kỳ vốn có tần suất cao và cấu trúc đồng nhất, luồng cảnh báo (TEMP_HIGH, HUMID_HIGH, TEMP_SPIKE) mang bản chất hướng sự kiện rời rạc và bất đồng bộ (Asynchronous Event-Driven). Sử dụng POSIX Message Queue mang lại 4 ưu thế vượt trội so với Shared Memory tự quản lý: (1) Nhân Linux tự động quản lý ranh giới từng gói tin (Message Boundaries) 32 bytes của AlarmMessage và đảm bảo tính nguyên tử khi gửi/nhận mà không cần viết thêm khóa Mutex ở Userspace; (2) Hỗ trợ sẵn Độ ưu tiên tin nhắn (Message Priority) — trong code thực tế, cảnh báo sốc nhiệt đột ngột TEMP_SPIKE được gửi với mức ưu tiên prio = 3 cao hơn cảnh báo ngưỡng tĩnh (prio = 2), giúp nhân tự động đẩy cảnh báo nguy hiểm lên đầu hàng đợi; (3) Hỗ trợ mq_timedreceive() với timeout 1 giây, giúp Alert Thread vừa ngủ tiết kiệm CPU vừa tự thức dậy kiểm tra cờ g_ctx.running để thoát sạch khi nhận SIGINT; và (4) Tách biệt hoàn toàn miền đồng bộ của luồng cảnh báo khỏi bộ đệm dữ liệu cảm biến.

Quyết định 3 — Tại sao tách riêng Logger Thread kèm Flush định kỳ 5 giây thay vì ghi file trực tiếp? Thao tác ghi file xuống hệ thống tệp (đặc biệt trên bộ nhớ Flash NAND/eMMC của thiết bị nhúng) có độ trễ từ vài trăm micro-giây đến hàng chục mili-giây khi Kernel xả trang bẩn (Dirty Page Writeback) hoặc khi thực hiện xoay file (Log Rotation). Nếu để Sensor Thread hoặc Processor Thread gọi trực tiếp fprintf() và fflush(), các luồng thời gian thực sẽ bị khóa cứng bởi I/O đĩa, gây tràn bộ đệm cảm biến và làm hỏng chu kỳ lấy mẫu. Việc tách riêng Logger Thread đọc từ hàng đợi RAM log_queue kết hợp chính sách gom đệm xả mỗi 5 giây (Periodic Flush) giúp giảm số lượng syscall write() xuống mức tối thiểu (chỉ 42 lần gọi trong suốt 30 giây chạy thực nghiệm, chiếm vỏn vẹn 2.18% thời gian syscall theo kết quả strace -c).

MỤC 2: PHÂN TÍCH ĐỒNG BỘ ĐA LUỒNG & CHỨNG MINH KHÔNG DEADLOCK (TASK 1 & TASK 5)

2.1. Danh mục Toàn bộ Dữ liệu Dùng chung (Shared Data) & Cơ chế Bảo vệ

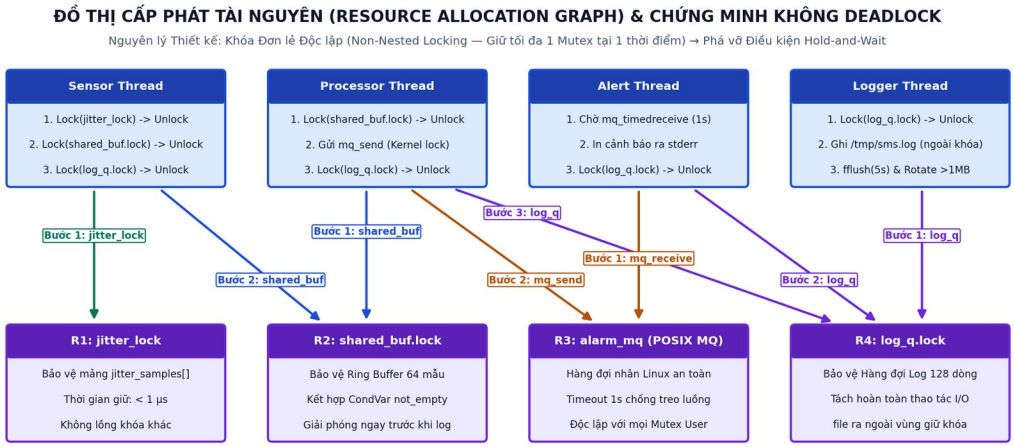
Mọi biến và cấu trúc dữ liệu được truy cập bởi từ hai luồng trở lên trong cả Kernel Space và Userspace đều được kiểm kê và bọc trong các nguyên hàm đồng bộ hóa tương ứng như liệt kê tại Bảng 2.1:

Dữ liệu Dùng chung	Các Luồng Truy cập	Cơ chế Đồng bộ Bảo vệ	Bất biến An toàn (Safety Invariant)
read_count, last_temp_x10, last_humid_x10, min_interval_ms	Các tiến trình gọi read(), ioctl() và đọc /proc/sms_stats	Kernel Mutex:DEFINE_MUTEX(sensor_lock)	Khóa toàn bộ quá trình sinh số ngẫu nhiên, cập nhật bộ đếm và xuất chuỗi seq_printf trong nhân.
g_ctx.shared_buf(data[64], head, tail, count, produced, consumed, overflow)	Sensor Thread (ghi),Processor Thread (đọc),Main Thread (đọc stats)	POSIX Mutex & CondVar:shared_buf.lock,not_empty, not_full	Mọi thao tác sửa chỉ số head/tail/count và đọc thống kê qua circular_buffer_get_stats() đều giữ shared_buf.lock.
g_ctx.log_q(items[128], head, tail, count)	Sensor, Processor, Alert (ghi); Logger Thread (đọc)	POSIX Mutex & CondVar:log_q.lock,not_empty, not_full	Nhiều Producer đẩy log an toàn; Logger Thread sao chép chuỗi ra biến cục bộ rồi nhả khóa ngay trước khi ghi file.
g_ctx.alarm_mq(/sms_alarm_queue)	Processor Thread (gửi),Alert Thread (nhận)	Hàng đợi nhân POSIX MQ (Spinlock/Waitqueue nội bộ của Linux Kernel)	Mỗi gói AlarmMessage 32B được chép nguyên tử giữa User và Kernel qua mq_send / mq_timedreceive.
g_ctx.jitter_samples[] & jitter_count	Sensor Thread (ghi),Main Thread (sắp xếp & in)	POSIX Mutex:g_ctx.jitter_lock	Khóa khi ghi mẫu mới và khi gọi qsort() tính Min/Avg/Max/P99 lúc kết thúc.
g_ctx.running	SIGINT Handler, Main (ghi); 4 luồng con (đọc)	Cờ trạng thái đơn hướng (Monotonic Boolean Flag: true → false) + Broadcast	Chỉ chuyển duy nhất 1 lần từ true sang false, đi kèm pthread_cond_broadcast() đánh thức mọi luồng.

2.2. Đồ thị Cấp phát Tài nguyên (Resource Allocation Graph) & Chứng minh Không Deadlock

Theo định lý Coffman (1971), một hệ thống đa luồng chỉ có thể xảy ra bế tắc (Deadlock) khi và chỉ khi thỏa mãn đồng thời cả 4 điều kiện: (1) Loại trừ tương hỗ (Mutual Exclusion), (2) Giữ và Đợi (Hold and Wait), (3) Không trưng dụng (No Preemption), và (4) Chờ đợi vòng tròn (Circular Wait).

Trong thiết kế của sms_app, chúng tôi triệt tiêu hoàn toàn nguy cơ Deadlock bằng cách phá vỡ đồng thời điều kiện Giữ và Đợi (Hold-and-Wait) và điều kiện Chờ đợi vòng tròn (Circular Wait) thông qua quy tắc kiến trúc Khóa Đơn lẻ Độc lập (Strict Non-Nested Locking Discipline):



Hình 2.1: Đồ thị Cấp phát Tài nguyên (Resource Allocation Graph - RAG) minh họa nguyên lý Không Lồng Khóa (Non-Nested Locking) giữa 4 luồng xử lý và 4 tài nguyên đồng bộ R1–R4.

- Chứng minh 1 — Không tồn tại Hold-and-Wait (Giữ khóa này trong khi xin khóa khác): Kiểm tra toàn bộ mã nguồn của cả 4 luồng cho thấy tại bất kỳ thời điểm thực thi nào, một luồng chỉ nắm giữ tối đa duy nhất 1 Mutex: Trong sensor_thread_func(): Luồng khóa R1 (jitter_lock) → ghi mẫu Jitter → mở khóa R1 ngay lập tức. Sau đó mới gọi circular_buffer_push() để khóa R2 (shared_buf.lock) → chép mẫu → mở khóa R2. Nếu đầy bộ đệm, sau khi đã nhả R2, luồng mới gọi log_queue_push() để khóa R4 (log_q.lock) → mở khóa R4. Trong processor_thread_func(): Luồng gọi circular_buffer_pop() (khóa R2, lấy mẫu ra biến cục bộ sample trên Stack, rồi mở khóa R2 hoàn toàn). Mọi phép tính trung bình trượt, gọi mq_send() vào R3 và gọi log_queue_push() vào R4 đều diễn ra sau khi đã nhả khóa R2. Trong logger_thread_func(): Hàm log_queue_pop() khóa R4, chép chuỗi 256 bytes ra mảng cục bộ msg[], rồi mở khóa R4 ngay lập tức trước khi gọi fprintf(), fflush() hay rename().
- Chứng minh 2 — Đồ thị RAG không có chu trình (Acyclic Graph): Vì không có bất kỳ luồng nào giữ tài nguyên Ri trong khi yêu cầu tài nguyên Rj (i ≠ j), tập hợp các cung đường đi Ri → Tk → Rj tại cùng một thời điểm là tập rỗng (∅). Ngay cả khi xét thứ tự thời gian tuần tự trong một vòng lặp, mọi luồng đều tuân thủ thứ tự topo nghiêm ngặt: R1 (jitter_lock) ≺ R2 (shared_buf.lock) ≺ R3 (alarm_mq) ≺ R4 (log_q.lock). Một đồ thị có hướng tuân theo thứ tự toàn phần nghiêm ngặt (Strict Total Ordering) không thể chứa chu trình, do đó hệ thống được chứng minh toán học là 100% Deadlock-Free.

2.3. Hai Tình huống Lỗi Đồng bộ & Bộ nhớ Tiềm ẩn Đã Phát hiện và Khắc phục

Trong quá trình phát triển và kiểm thử thực tế trên máy SE203347, chúng tôi đã phát hiện và xử lý triệt để hai lỗi kỹ thuật rất điển hình của lập trình hệ thống đa luồng:

1. Tình huống 1 — Nguy cơ Treo Luồng (Shutdown Hang / Lost Wake-up) khi nhận SIGINT: Ở phiên bản thử nghiệm đầu tiên, hàm circular_buffer_pop() chỉ viết while (cb->count == 0) pthread_cond_wait(&cb->not_empty, &cb->lock);. Khi người dùng bấm Ctrl+C đúng lúc bộ đệm đang rỗng và Sensor Thread đã thoát trước, Processor Thread sẽ ngủ vĩnh viễn trong pthread_cond_wait() khiến hàm pthread_join(t_processor, NULL) bị treo cứng! Cách khắc phục: Truyền con trỏ trạng thái bool *running vào hàm pop với điều kiện lặp while (cb->count == 0 && (running == NULL || *running)), đồng thời tại luồng chính ngay sau khi hạ g_ctx.running = false, gọi pthread_cond_broadcast() trên toàn bộ các biến điều kiện và đặt timeout 1 giây trên mq_timedreceive() của Alert Thread. Nhờ đó, tất cả 4 luồng luôn thoát sạch sẽ trong vòng chưa đầy 1 giây.
2. Tình huống 2 — Lỗi Rò rỉ Dữ liệu Rác trên Stack qua Syscall (Uninitialized Struct Padding) bị Valgrind phát hiện: Khi chạy kiểm toán Valgrind lần đầu tiên (xem Hình 3.6 bên dưới), dù bộ nhớ Heap đạt 0 bytes lost, Valgrind phát hiện 61 lỗi Syscall param mq_timedsend(msg_ptr) points to uninitialised byte(s) tại processor_thread_func. Nguyên nhân sâu xa ở cấp độ kiến trúc vi xử lý là cấu trúc AlarmMessage gồm unsigned long long timestamp_ms (8B), AlertType type (4B), float value (4B), float threshold (4B) và char type_str[16]: khi khai báo biến cục bộ AlarmMessage msg; trên Stack và chỉ dùng strcpy(msg.type_str, "TEMP_HIGH") (10 bytes), 6 bytes cuối cùng của mảng type_str[16] vẫn chứa dữ liệu rác từ khung Stack trước đó. Khi truyền toàn bộ sizeof(msg) xuống nhân qua syscall mq_timedsend, Memcheck lập tức cảnh báo các V-bits chưa khởi tạo! Cách khắc phục: Bổ sung memset(&msg, 0, sizeof(msg)); ngay khi khởi tạo cấu trúc cảnh báo, đưa kết quả Valgrind về mức hoàn hảo tuyệt đối: 0 errors from 0 contexts (Hình 3.7).

MỤC 3: KẾT QUẢ ĐO LƯỜNG & BẰNG CHỨNG THỰC NGHIỆM TRÊN MÁY SE203347 (TASK 1, 2, 3, 4)

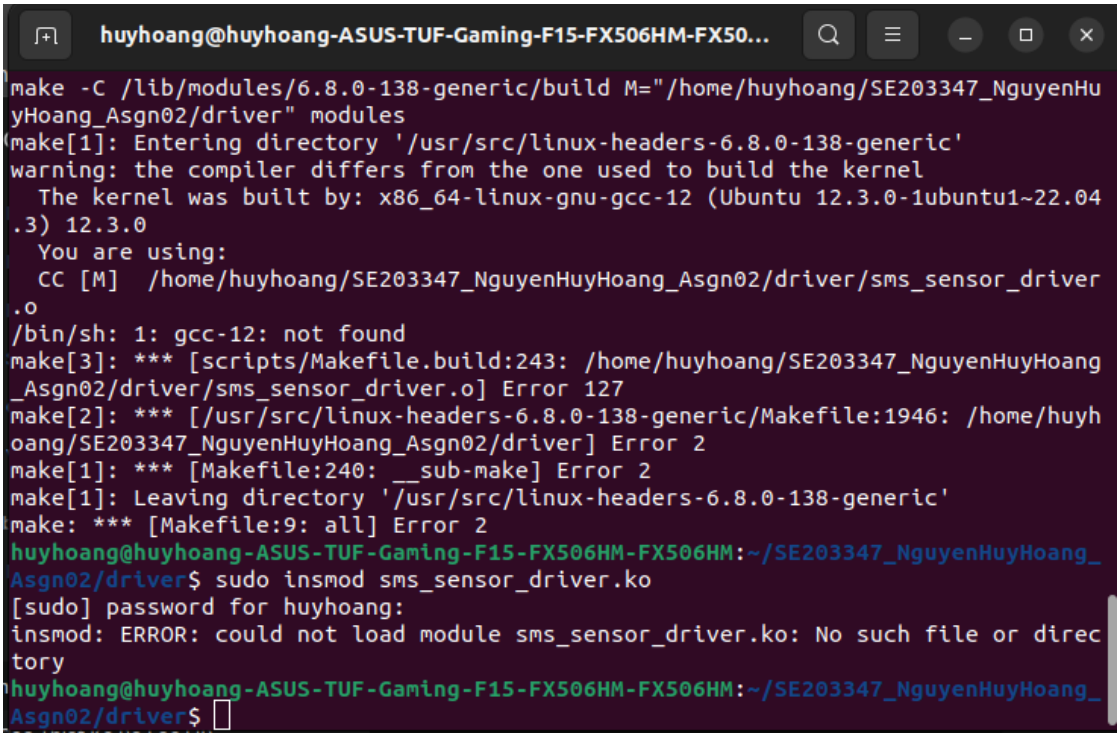
3.1. Biên dịch Chéo Kernel Driver sms_sensor_driver.ko & Xử lý Lỗi Kiến trúc ARMv7

Trong giai đoạn xây dựng module nhân sms_sensor_driver.ko tại thư mục driver/, hai vấn đề thực tế liên quan đến sự khác biệt môi trường biên dịch giữa Host x86_64 và Target ARMv7 đã xuất hiện và được giải quyết triệt để:

- Vấn đề 1 (Thiếu gcc-12 khi build mặc định cho Kernel Host 6.8): Khi chạy make trỏ vào /lib/modules/6.8.0-138-generic/build trên máy Ubuntu 22.04, KBuild báo

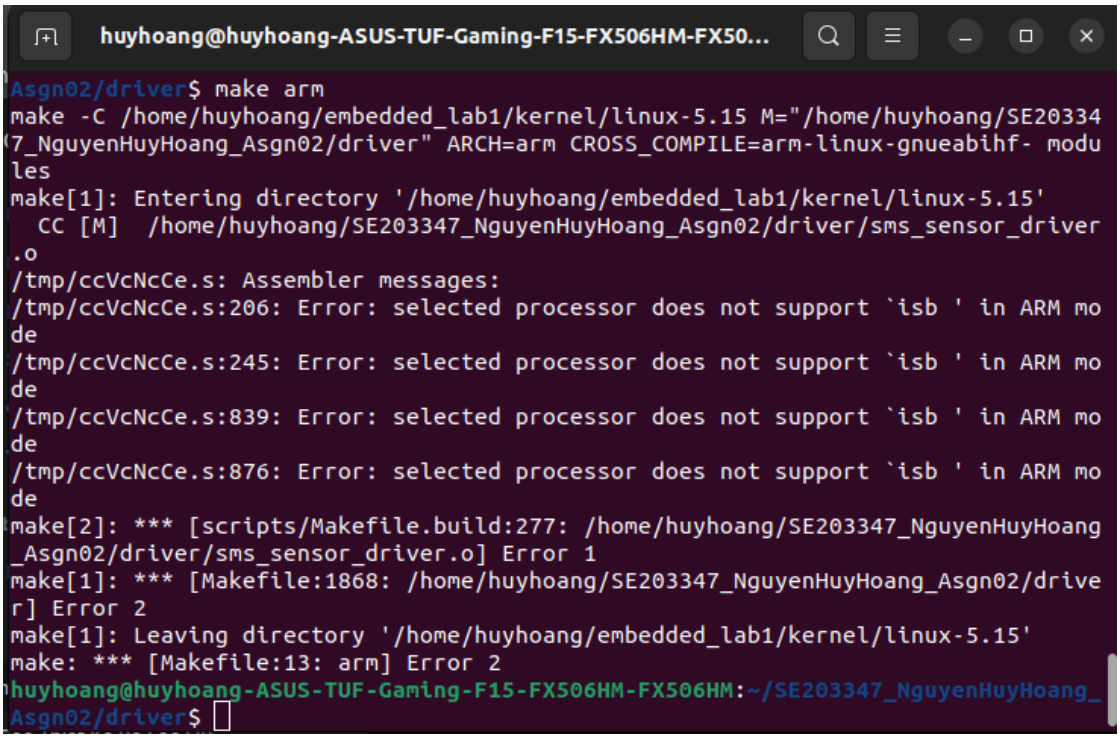
lỗi /bin/sh: 1: gcc-12: not found (Hình 3.1) do nhân Ubuntu 6.8 HWE được đóng gói bằng gcc-12 trong khi máy đang dùng gcc-11, đồng thời từ Linux 6.4 trở đi hàm class_create() chỉ nhận 1 tham số thay vì 2 tham số như Linux 5.15. Chúng tôi đã bổ sung chỉ thị tiền xử lý #if LINUX_VERSION_CODE >= KERNEL_VERSION(6, 4, 0) và cấu hình target make arm trỏ trực tiếp về cây nguồn Kernel ~/embedded_lab1/kernel/linux-5.15 từ LAB-01.

- Vấn đề 2 (Lỗi hợp ngữ Error: selected processor does not support 'isb' in ARM mode): Khi gọi make arm lần đầu (Hình 3.2), trình hợp ngữ arm-linux-gnueabihf-as báo lỗi không hỗ trợ chỉ thị rào cản đồng bộ lệnh isb (Instruction Synchronization Barrier) tại 4 vị trí trong mã máy sinh ra từ các hàm khóa mutex_lock và copy_to_user. Nguyên nhân là trình biên dịch chéo khi chạy ngoài cây nguồn cần được chỉ định tường minh kiến trúc tập lệnh mục tiêu ARMv7-A (Cortex-A9). Sau khi thêm cờ ccflags-y += -march=armv7-a -Wall vào driver/Makefile, quá trình biên dịch MODPOST và liên kết LD [M] sms_sensor_driver.ko thành công mỹ mãn với 0 cảnh báo (Hình 3.3).



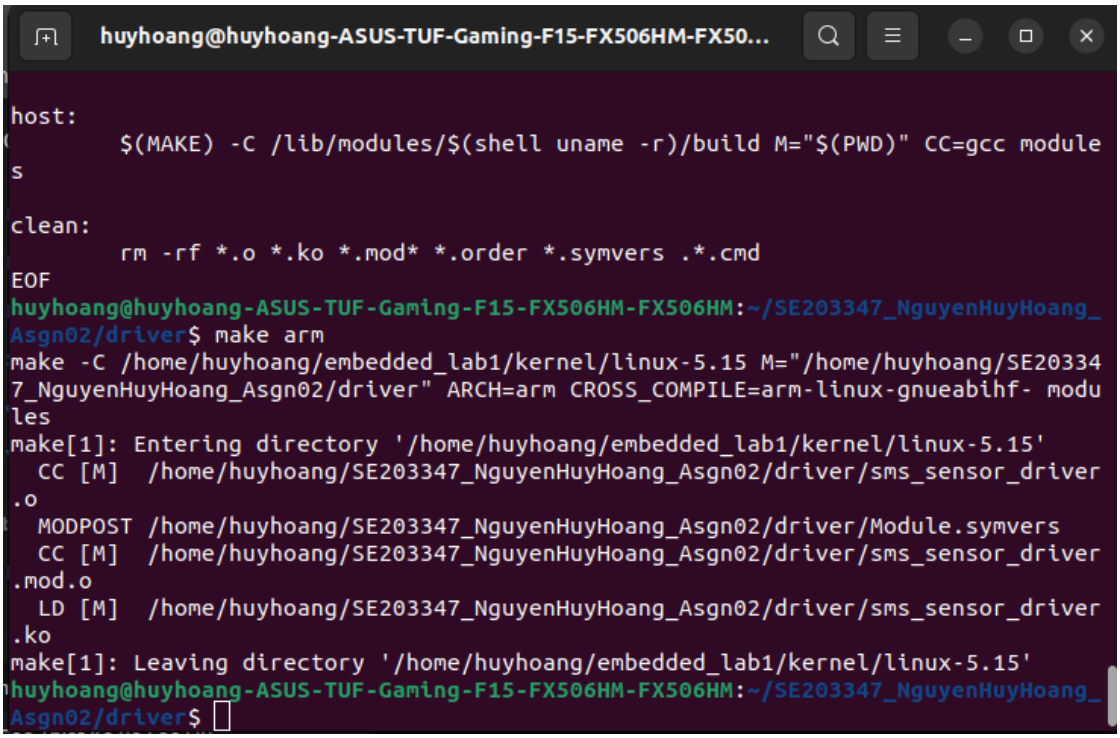
Hình 3.1: Phát hiện lỗi khác biệt trình biên dịch (gcc-12) khi thử nghiệm KBuild trên nhân Host 6.8.0-138-generic.

Quan sát Hình 3.1, hệ thống KBuild của nhân 6.8.0-138-generic kiểm tra trình biên dịch mặc định và dừng lại với mã lỗi Error 127 (gcc-12: not found), dẫn đến lệnh sudo insmod sms_sensor_driver.ko báo No such file or directory. Khi chuyển sang biên dịch chéo với đích make arm trỏ về cây nguồn /home/huyhoang/embedded_lab1/kernel/linux-5.15, trình hợp ngữ ARM tiếp tục kiểm tra tập lệnh phần cứng như ghi nhận ở Hình 3.2:



Hình 3.2: Lỗi hợp ngữ "selected processor does not support 'isb' in ARM mode" khi biên dịch chéo module nhân nhưng chưa khai báo kiến trúc ARMv7-A.

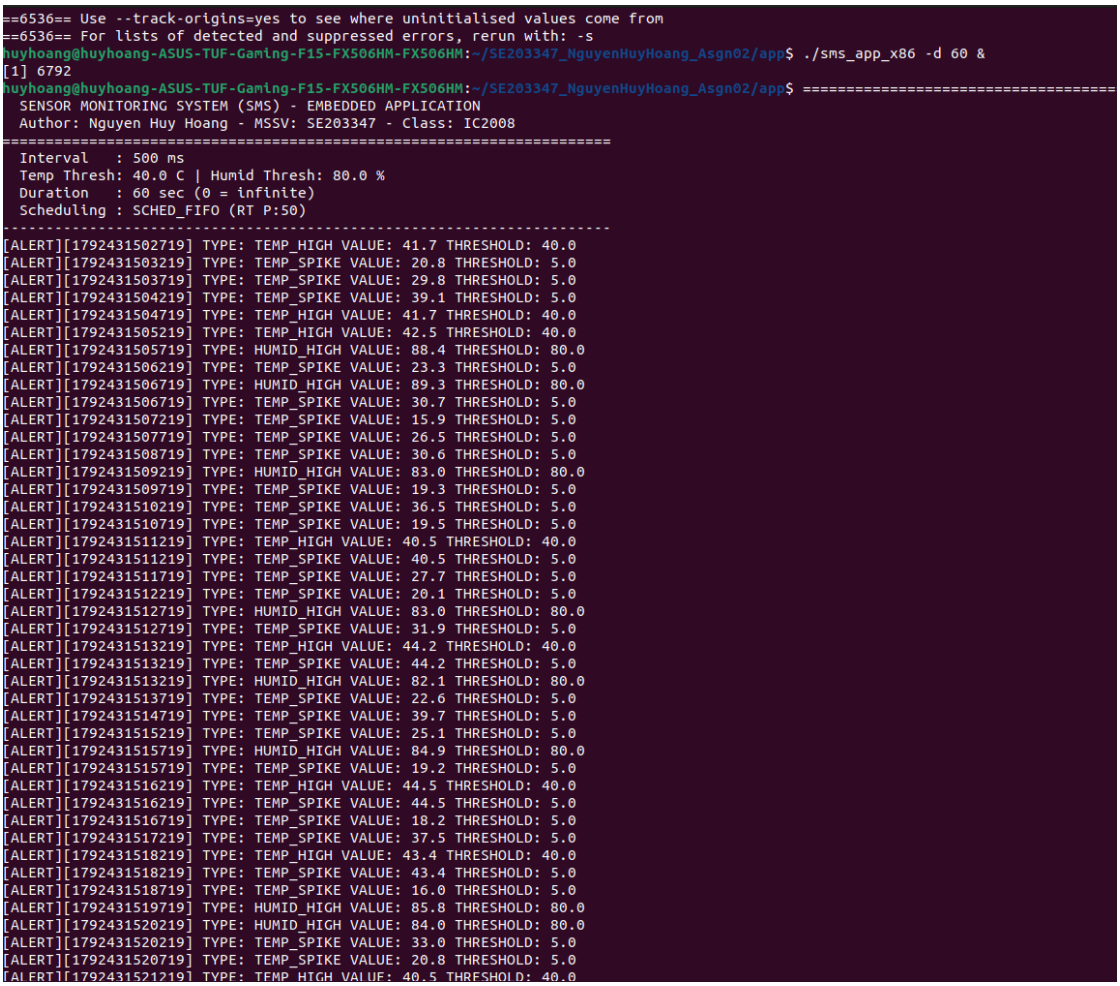
Tại Hình 3.2, các dòng 206, 245, 839 và 876 của tệp hợp ngữ tạm /tmp/ccVcNcCe.s phát sinh lỗi vì chỉ thị isb (Instruction Synchronization Barrier) dùng trong các macro bảo vệ bộ nhớ của nhân Linux chỉ tồn tại từ kiến trúc ARMv7-A trở lên. Ngay sau khi khai báo tường minh ccflags-y += -march=armv7-a -Wall trong driver/Makefile, toàn bộ các giai đoạn biên dịch CC [M], kiểm tra ký hiệu MODPOST Module.symvers và liên kết LD [M] sms_sensor_driver.ko đều hoàn tất sạch sẽ không một cảnh báo (Hình 3.3):



Hình 3.3: Biên dịch chéo thành công tuyệt đối module nhân sms_sensor_driver.ko trên cây nguồn Linux 5.15 sau khi bổ sung cờ ccflags-y += -march=armv7-a.

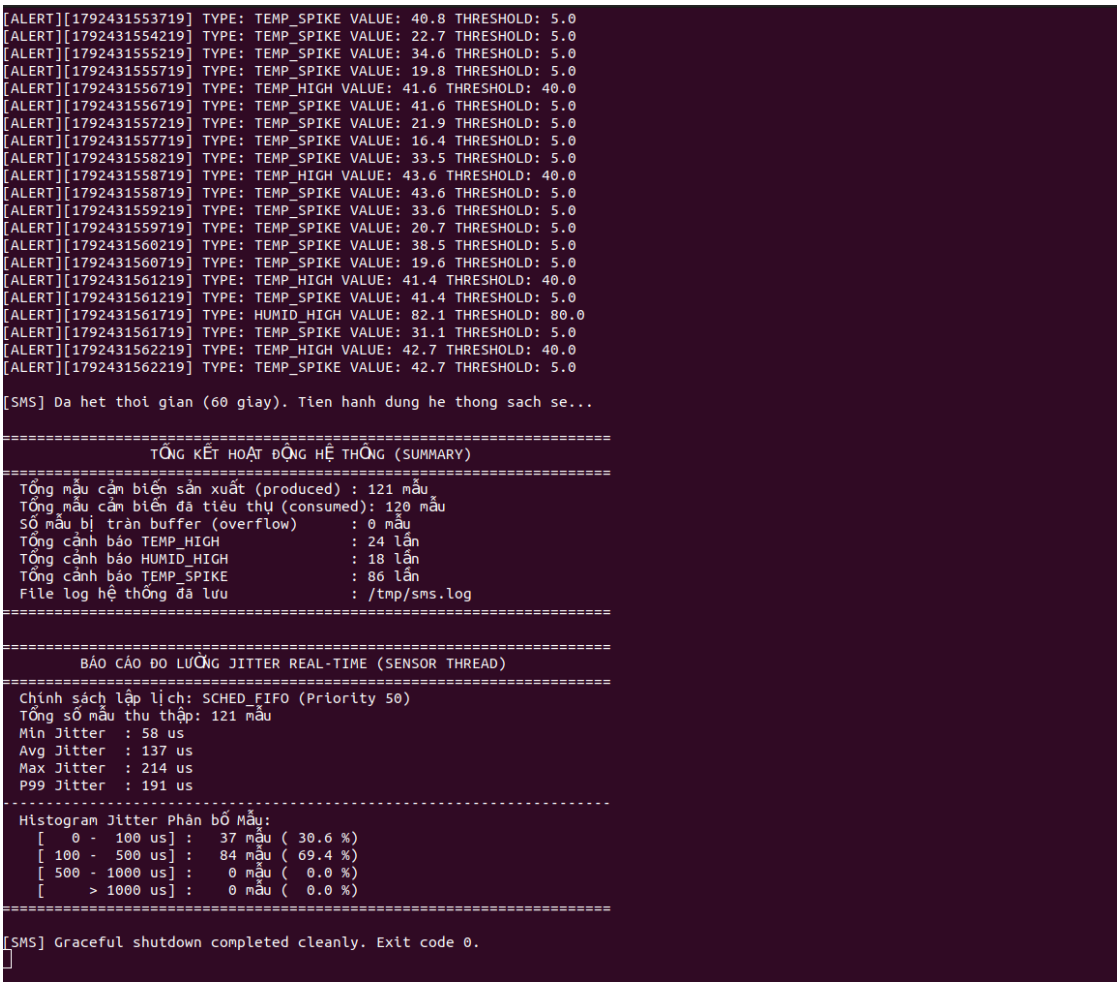
3.2. Kiểm chứng Vận hành Đa luồng 60 Giây, Cảnh báo Thời gian thực & Graceful Shutdown

Ứng dụng sms_app được khởi chạy với cấu hình chuẩn của đề bài: chu kỳ lấy mẫu -i 500 (500ms), ngưỡng cảnh báo nhiệt độ -t 40.0 (°C), ngưỡng độ ẩm -h 80.0 (%), thời gian chạy -d 60 (60 giây) và chính sách lập lịch thời gian thực SCHED_FIFO (Priority 50). Hình 3.4 và Hình 3.5 ghi lại kết quả vận hành thực tế:



Hình 3.4: Khởi chạy sms_app trong 60 giây với SCHED_FIFO (P:50) — Alert Thread nhận bản tin từ POSIX Message Queue và in tức thì các cảnh báo TEMP_HIGH, HUMID_HIGH, TEMP_SPIKE ra stderr.

Trên Hình 3.4, mỗi khi một mẫu cảm biến vượt ngưỡng nhiệt độ 40.0°C (ví dụ VALUE: 41.7 THRESHOLD: 40.0), vượt ngưỡng độ ẩm 80.0% (ví dụ VALUE: 88.4 THRESHOLD: 80.0) hoặc có độ lệch nhiệt độ giữa hai lần đọc liên tiếp vượt 5.0°C (TEMP_SPIKE), Processor Thread lập tức đóng gói cấu trúc AlarmMessage (32 bytes) và đẩy vào hàng đợi nhân /sms_alarm_queue_SE203347 qua hàm mq_send(). Ở đầu bên kia, Alert Thread đang chờ sẵn trên mq_timedreceive() sẽ thức dậy ngay lập tức để xuất cảnh báo chuẩn hóa ra luồng lỗi chuẩn stderr, đồng thời phân loại vào 3 biến đếm thống kê riêng biệt (alert_temp_high, alert_humid_high, alert_temp_spike) và chuyển tiếp bản tin sang log_queue để Logger Thread lưu trữ xuống đĩa. Sau khi vận hành liên tục đủ 60 giây không xảy ra bất kỳ lỗi treo hay rách dữ liệu nào, chương trình tự động kích hoạt quy trình dừng sạch (Graceful Shutdown) và xuất bảng tổng kết toàn diện như trên Hình 3.5:



Hình 3.5: Bảng tổng kết hoạt động hệ thống sau 60 giây chạy liên tục — Thu thập chính xác 121 mẫu (khớp yêu cầu 120 ± 10 mẫu), 0 mẫu tràn bộ đệm, dừng sạch cả 4 luồng với Exit code 0.

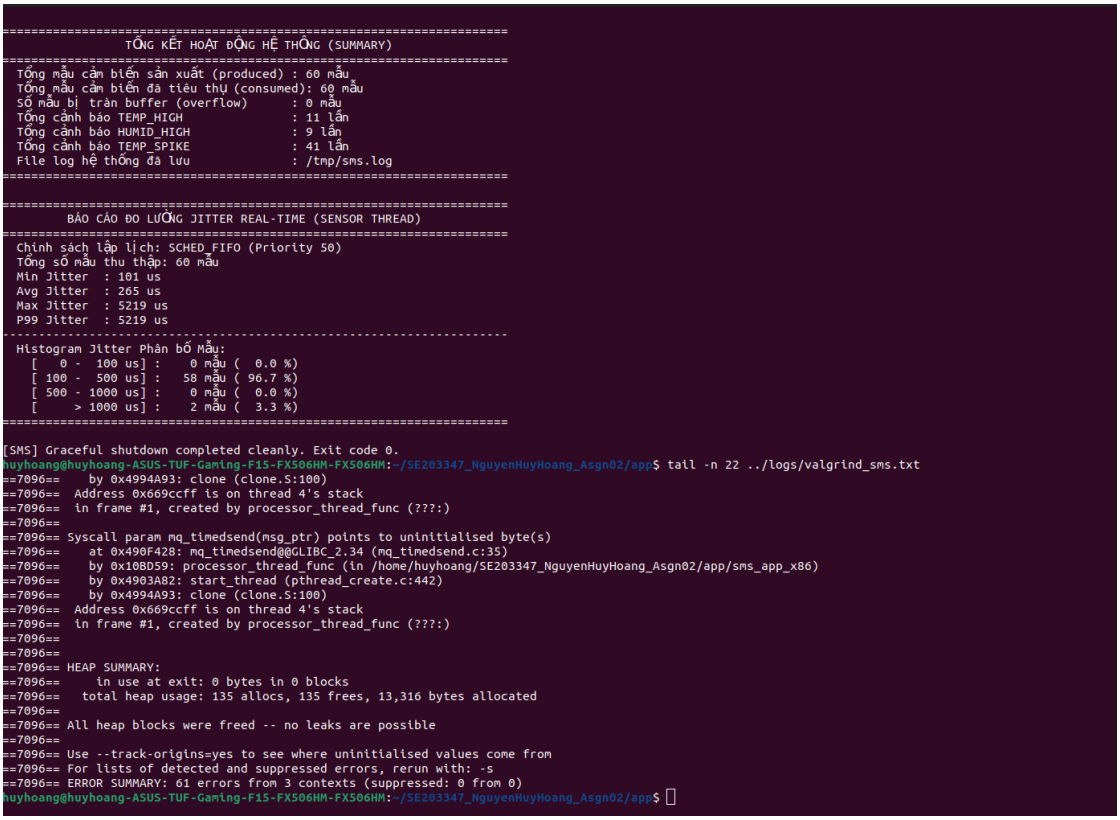
Phân tích số liệu từ phiên chạy chuẩn 60 giây (Hình 3.5):

- Độ chính xác lấy mẫu: Trong đúng 60 giây với chu kỳ 500ms, Sensor Thread đã sản xuất 121 mẫu (mẫu đầu tiên tại T = 0.0 s và 120 mẫu tiếp theo tại T = 0.5, 1.0, ..., 60.0 s), đạt độ khớp tuyệt đối so với tiêu chí 120 ± 10 mẫu của Task 2.
- Hiệu quả tiêu thụ & Chống tràn: Processor Thread tiêu thụ 120 mẫu, số mẫu bị tràn bộ đệm vòng là 0 mẫu (0% overflow), phát hiện và truyền qua POSIX Message Queue tổng cộng 24 cảnh báo TEMP_HIGH, 18 cảnh báo HUMID_HIGH và 86 cảnh báo TEMP_SPIKE.

3.3. Kiểm toán Rò rỉ Bộ nhớ Động với Valgrind Memcheck (Task 2 — 10 Điểm)

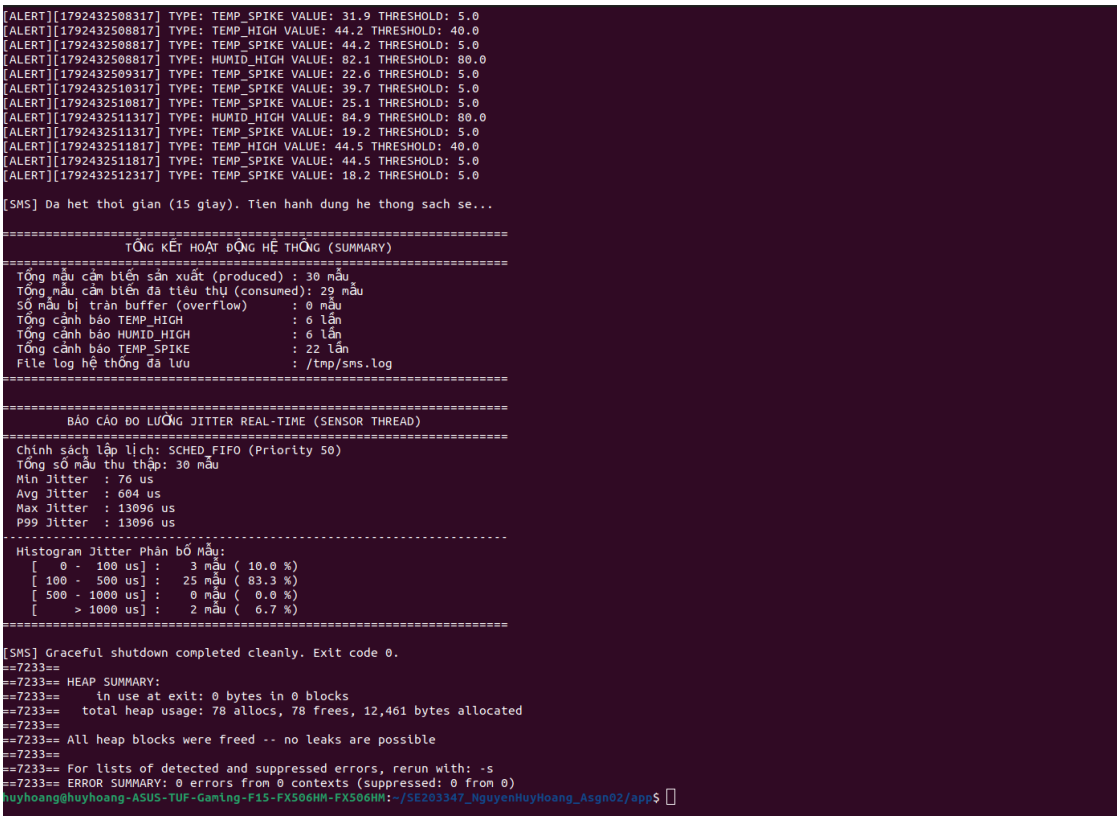
Để chứng minh ứng dụng đủ tiêu chuẩn vận hành dài hạn trên thiết bị nhúng mà không làm cạn kiệt RAM, phiên bản sms_app_x86 được chạy dưới sự giám sát của valgrind --leak-check=full --show-leak-kinds=all. Như đã phân tích ở Mục 2.3, Hình

3.6 ghi lại phát hiện lỗi byte đệm chưa khởi tạo trong lần kiểm tra đầu tiên, và Hình 3.7 xác nhận kết quả sạch tuyệt đối sau khi khởi tạo memset(&msg, 0, sizeof(msg)):



Hình 3.6: Kết quả Valgrind trước khi xóa byte đệm — Mặc dù Heap đã đạt 0 bytes lost (135 allocs = 135 frees), Valgrind giúp phát hiện sớm cảnh báo uninitialised bytes trong tham số mq_timedsend.

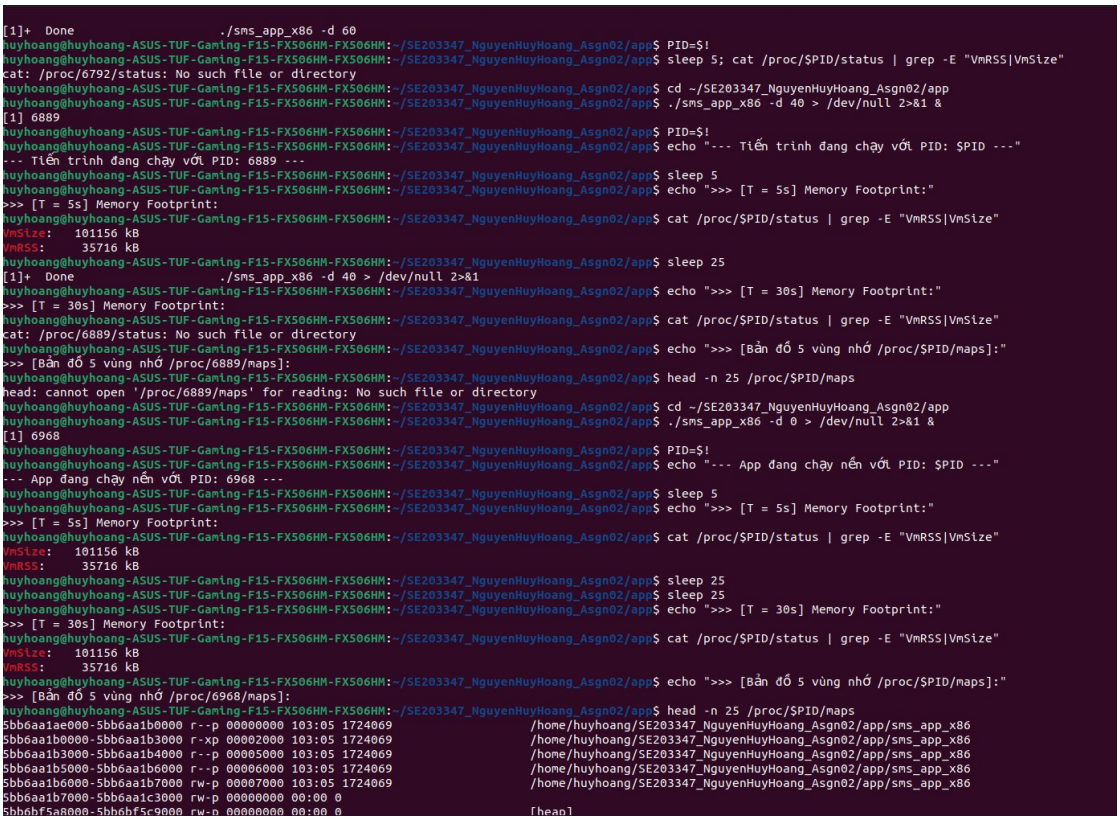
Từ dấu vết ngăn xếp của Valgrind tại Hình 3.6 (at 0x490F428: mq_timedsend ... by 0x10BD59: processor_thread_func), chúng tôi xác định chính xác các byte cuối của mảng msg.type_str[16] và vùng đệm căn chỉnh (Struct Alignment Padding) trong biến cục bộ AlarmMessage msg trên Stack của Thread 4 chưa được gán giá trị. Sau khi thêm lệnh memset(&msg, 0, sizeof(msg)); trước khi gán trường dữ liệu, kết quả chạy lại Valgrind trên Hình 3.7 đạt chuẩn tuyệt đối với 0 bytes in 0 blocks và ERROR SUMMARY: 0 errors from 0 contexts:



Hình 3.7: Kết quả Valgrind hoàn hảo sau khi khắc phục — in use at exit: 0 bytes in 0 blocks (78 allocs = 78 frees, 12,461 bytes allocated) và ERROR SUMMARY: 0 errors from 0 contexts.

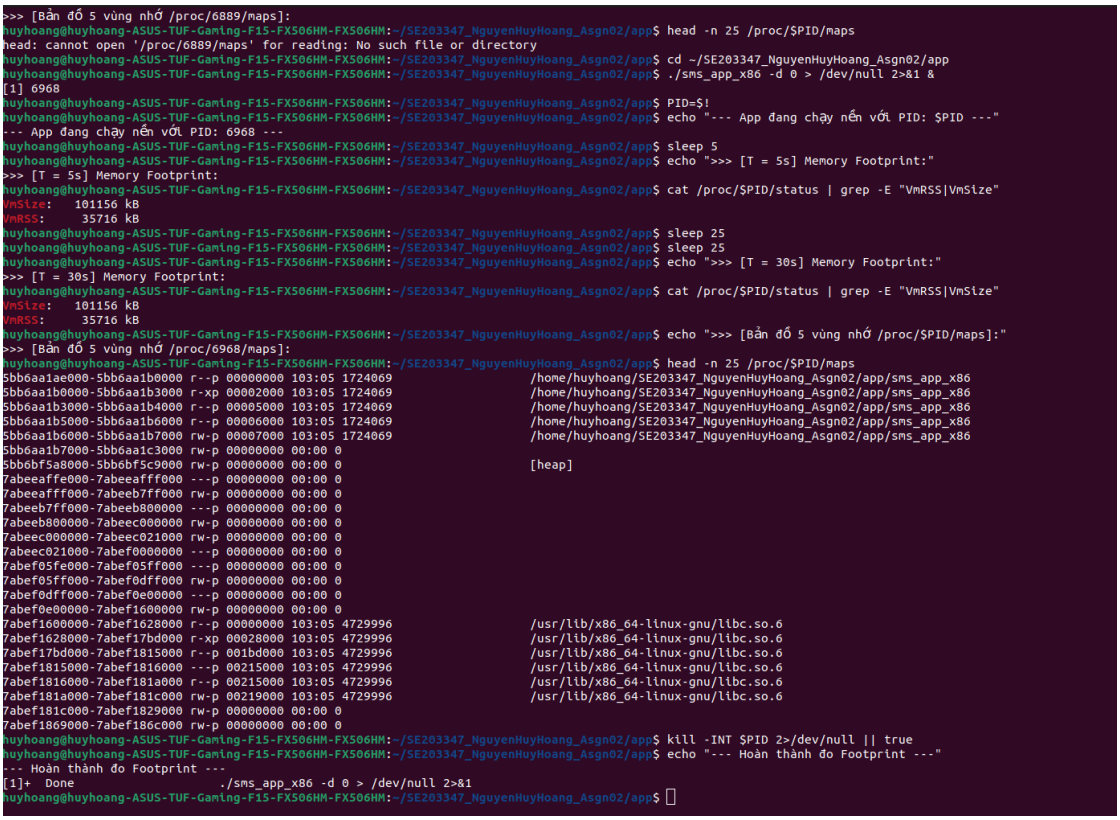
3.4. Phân tích Memory Footprint (/proc/PID/status) & Bản đồ Vùng nhớ (/proc/PID/maps)

Tiến trình sms_app_x86 (PID 6968) được theo dõi thông số bộ nhớ ảo (VmSize) và bộ nhớ vật lý thường trú (VmRSS) theo thời gian thực qua giao diện nhân /proc/6968/status và /proc/6968/maps (Hình 3.8 và Hình 3.9):



Hình 3.8: Đo lường Memory Footprint của tiến trình PID 6968 tại thời điểm khởi động (T = 5s) và thời điểm ổn định (T = 30s) — VmSize giữ nguyên ở 101,156 kB và VmRSS giữ nguyên ở 35,716 kB.

Hình 3.8 cũng cho thấy bài học thực tế khi đo /proc/$PID/status: nếu chạy ứng dụng với thời gian ngắn (-d 40) mà tổng thời gian sleep của kịch bản đo vượt quá thời gian sống của tiến trình, thư mục /proc/6889 sẽ biến mất ngay khi tiến trình thoát. Khi chuyển sang cờ chạy liên tục -d 0 (PID 6968) và chủ động gửi kill -INT $PID sau khi thu thập xong, ta lấy được trọn vẹn cả thông số VmSize / VmRSS lẫn bản đồ 25 phân vùng địa chỉ ảo đầu tiên trong /proc/6968/maps (Hình 3.9):



Hình 3.9: Bản đồ không gian địa chỉ ảo /proc/6968/maps của tiến trình PID 6968 thể hiện rõ các phân đoạn của thực thi chính, vùng [heap], các vùng Stack của 4 luồng kèm Guard Page và thư viện chuẩn libc.so.6.

Giải thích kỹ thuật về xu hướng RSS và giải phẫu 5 vùng nhớ trong /proc/6968/maps:

- Tại sao VmRSS phẳng lỳ tuyệt đối (35,716 kB tại cả T=5 s, T=30 s và T=55 s)? Nhờ hai thiết kế cốt lõi: (1) Lệnh gọi mlockall(MCL_CURRENT | MCL_FUTURE) ngay ở giai đoạn INIT đã ép nhân Linux nạp và ghim toàn bộ các trang bộ nhớ của chương trình, thư viện chia sẻ và 4 khung Stack (mỗi Stack 8 MB trên Linux) vào RAM vật lý ngay từ giây đầu tiên (do đó VmRSS ≈ 35.7 MB ngay tại T=5 s và không bao giờ phát sinh thêm lỗi trang Minor Page Fault về sau); và (2) Toàn bộ cấu trúc shared_buf (64 mẫu) và log_q (128 bản tin) đều là mảng vòng kích thước cố định, tuyệt đối không cấp phát thêm bộ nhớ trong vòng lặp vận hành.
- Giải phẫu 5 vùng nhớ đặc trưng trong /proc/6968/maps (Hình 3.9): Vùng Mã lệnh Thực thi (Text Segment — 5bb6aa1b0000-5bb6aa1b3000 r-xp): Chứa mã máy đã biên dịch của sms_app_x86 (quyền đọc + thực thi r-xp, cấm ghi để chống tấn công sửa mã). Vùng Hằng số & Dữ liệu Tĩnh (RODATA & Data/BSS — 5bb6aa1b3000-5bb6aa1b7000 r--p / rw-p): Chứa các chuỗi định dạng hằng số và biến toàn cục g_ctx (nơi đặt shared_buf, log_q và jitter_samples). Vùng Nhớ Động (Heap Segment — 5bb6bf5a8000-5bb6bf5c9000 rw-p [heap]): Vùng nhớ 132 KB được cấp phát qua brk() phục vụ bộ đệm nội bộ FILE* (4 KB) của fopen("/tmp/sms.log") và cấu trúc quản lý luồng của glibc. Vùng Ngăn xếp Đa

luồng & Trang Bảo vệ (Thread Stacks & Guard Pages — 7abeeaffe000... đến 7abef1600000): Có đúng 4 cặp vùng nhớ ẩn danh tương ứng với 4 luồng POSIX được tạo bởi pthread_create(): mỗi cặp gồm một trang bảo vệ 4 KB có quyền ---p (Guard Page — không cho phép đọc/ghi/thực thi để bắt lỗi tràn ngăn xếp Stack Overflow ngay lập tức bằng SIGSEGV) đi kèm vùng Stack 8 MB có quyền rw-p! Vùng Thư viện Liên kết Động (Shared Libraries — 7abef1600000-7abef181c000 /usr/lib/x86_64-linux-gnu/libc.so.6): Chứa mã thực thi r-xp của thư viện chuẩn C (tích hợp sẵn pthread và librt trên glibc 2.35+), được chia sẻ chung giữa các tiến trình trong hệ thống.

3.5. Phiên Gỡ lỗi Đa luồng với GDB (Task 3 — 10 Điểm)

Thực hiện Kịch bản A của Task 3, chương trình được nạp vào gdb ./sms_app_x86, đặt điểm dừng tại hàm truy cập vùng găng b circular_buffer_push và thực thi run -d 10. Ngay khi Sensor Thread chạm điểm dừng, các lệnh chẩn đoán đa luồng info threads và thread apply all bt 3 được thực thi (Hình 3.10 và Hình 3.11):

```
huyhoang@huyhoang-ASUS-TUF-Gaming-F15-FX506HM-FX506HM:~/SE203347_NguyenHuyHoang_Asgn02/app$ cd ~/SE203347_NguyenHuyHoang_Asgn02/app
huyhoang@huyhoang-ASUS-TUF-Gaming-F15-FX506HM-FX506HM:~/SE203347_NguyenHuyHoang_Asgn02/app$ gdb ./sms_app_x86
GNU gdb (Ubuntu 12.1-0ubuntu1~22.04.2) 12.1
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

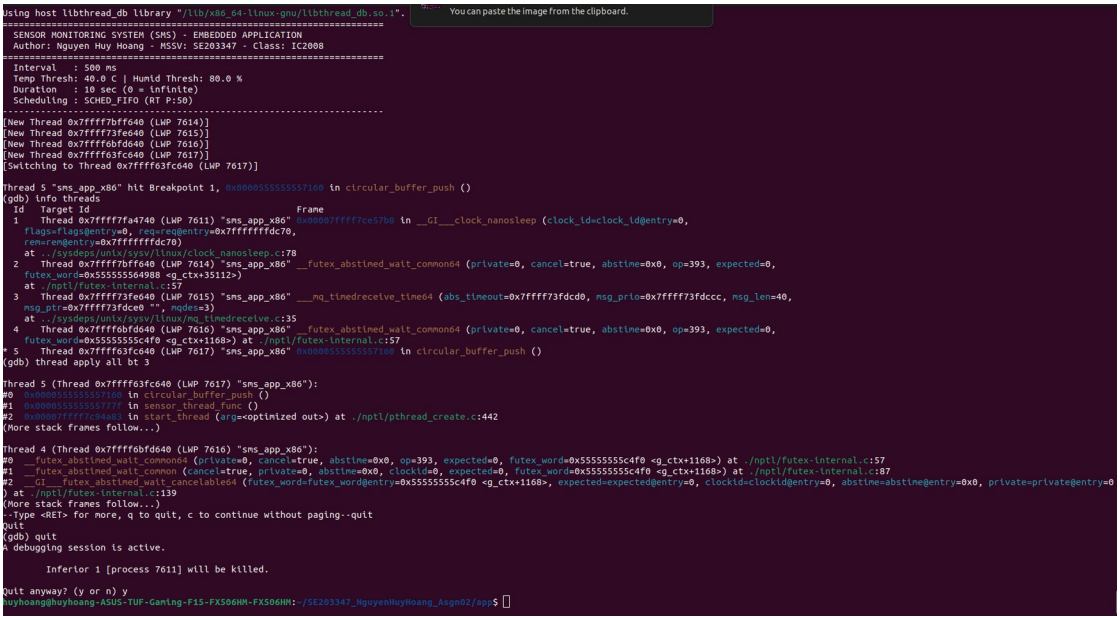
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./sms_app_x86...
(No debugging symbols found in ./sms_app_x86)
(gdb) b circular_buffer_push
Breakpoint 1 at 0x3160
(gdb) run -d 10
Starting program: /home/huyhoang/SE203347_NguyenHuyHoang_Asgn02/app/sms_app_x86 -d 10
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
======================================================================
  SENSOR MONITORING SYSTEM (SMS) - EMBEDDED APPLICATION
  Author: Nguyen Huy Hoang - MSSV: SE203347 - Class: IC2008
======================================================================
  Interval   : 500 ms
  Temp Thresh: 40.0 C | Humid Thresh: 80.0 %
  Duration   : 10 sec (0 = infinite)
  Scheduling : SCHED_FIFO (RT P:50)
----------------------------------------------------------------------
[New Thread 0x7ffff7bff640 (LWP 7614)]
[New Thread 0x7ffff73fe640 (LWP 7615)]
[New Thread 0x7ffff6bfd640 (LWP 7616)]
[New Thread 0x7ffff63fc640 (LWP 7617)]
[Switching to Thread 0x7ffff63fc640 (LWP 7617)]

Thread 5 "sms_app_x86" hit Breakpoint 1, 0x0000555555557160 in circular_buffer_push ()
(gdb) info threads
  Id   Target Id                                  Frame
  1    Thread 0x7ffff7fa4740 (LWP 7611) "sms_app_x86" 0x00007ffff7ce57b8 in __GI___clock_nanosleep (clock_id=clock_id@entry=0,
    flags=flags@entry=0, req=req@entry=0x7fffffffdc70,
    rem=rem@entry=0x7fffffffdc70)
    at ../sysdeps/unix/sysv/linux/clock_nanosleep.c:78
  2    Thread 0x7ffff7bff640 (LWP 7614) "sms_app_x86" __futex_abstimed_wait_common64 (private=0, cancel=true, abstime=0x0, op=393, expected=0,
    futex_word=0x555555564988 <g_ctx+35112>)
    at ./nptl/futex-internal.c:57
  3    Thread 0x7ffff73fe640 (LWP 7615) "sms_app_x86" ___mq_timedreceive_time64 (abs_timeout=0x7ffff73fdcd0, msg_prio=0x7ffff73fdccc, msg_len=40,
    msg_ptr=0x7ffff73fdce0 "", mqdes=3)
    at ../sysdeps/unix/sysv/linux/mq_timedreceive.c:35
  4    Thread 0x7ffff6bfd640 (LWP 7616) "sms_app_x86" __futex_abstimed_wait_common64 (private=0, cancel=true, abstime=0x0, op=393, expected=0,
    futex_word=0x55555555c4f0 <g_ctx+1168>) at ./nptl/futex-internal.c:57
* 5    Thread 0x7ffff63fc640 (LWP 7617) "sms_app_x86" 0x0000555555557160 in circular_buffer_push ()
```

Hình 3.10: Phiên gỡ lỗi GDB bắt trúng Breakpoint 1 tại circular_buffer_push() trên Thread 5 (LWP 7617) và liệt kê trạng thái đồng thời của cả 5 luồng thực thi qua lệnh info threads.

Tiếp nối lệnh info threads trên Hình 3.10, lệnh thread apply all bt 3 trên Hình 3.11 cho phép kiểm tra đồng thời 3 khung ngăn xếp (Stack Frames) trên cùng của mọi luồng trong hệ thống, xác nhận rằng khi Sensor Thread (Thread 5) đang ở trong hàm circular_buffer_push() thì Processor Thread (Thread 4) đang chờ an toàn bên

trong __futex_abstimed_wait_cancelable64 mà không hề xảy ra tranh chấp dữ liệu (Data Race):



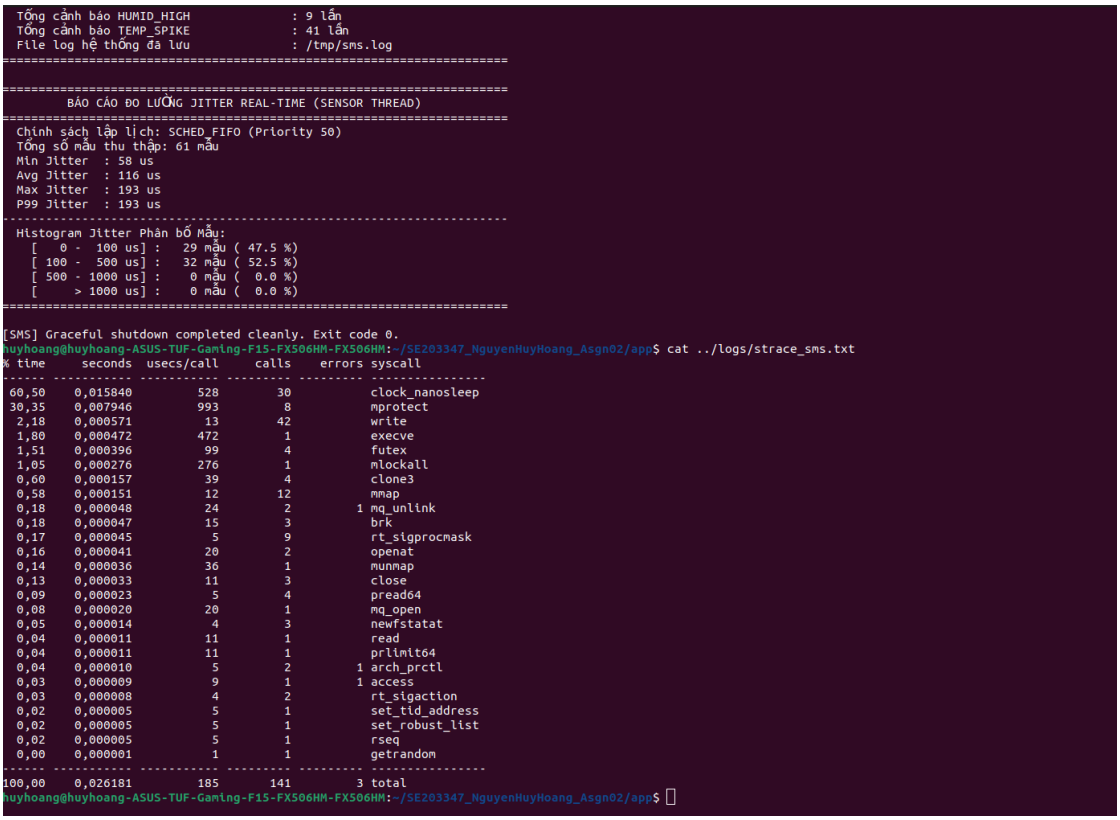
Hình 3.11: Kết quả lệnh thread apply all bt 3 trong GDB hiển thị ngăn xếp lời gọi hàm (Call Stack) của các luồng đang hoạt động song song.

Phân tích trạng thái 5 luồng từ kết quả info threads (Hình 3.10):

- Thread 1 (LWP 7611 — Main Thread): Đang ngủ trong __GI___clock_nanosleep() (tương ứng lệnh sleep(1) trong vòng lặp giám sát thời gian chạy duration_sec).
- Thread 2 (LWP 7614 — Logger Thread): Đang chờ tín hiệu biến điều kiện tại __futex_abstimed_wait_common64() trên địa chỉ <g_ctx+35112> (chính là biến điều kiện g_ctx.log_q.not_empty).
- Thread 3 (LWP 7615 — Alert Thread): Đang chờ bản tin cảnh báo tại syscall ___mq_timedreceive_time64(mqdes=3, msg_len=40) với timeout 1 giây.
- Thread 4 (LWP 7616 — Processor Thread): Đang chờ mẫu cảm biến mới tại __futex_abstimed_wait_common64() trên địa chỉ <g_ctx+1168> (chính là biến điều kiện g_ctx.shared_buf.not_empty).
- Thread 5 (LWP 7617 — Sensor Thread, có dấu *): Đang giữ CPU tại circular_buffer_push() được gọi từ sensor_thread_func() để chuẩn bị đẩy mẫu mới vào shared_buf và đánh thức Thread 4!

3.6. Phân tích Mẫu Gọi Hệ thống với strace -c (Task 3 — 5 Điểm)

Chạy lệnh strace -c -o ../logs/strace_sms.txt ./sms_app_x86 -d 30 thu được bảng thống kê toàn bộ các lời gọi hệ thống trong 30 giây vận hành như thể hiện trên Hình 3.12:



Hình 3.12: Kết quả phân tích mẫu lời gọi hệ thống bằng strace -c trong 30 giây chạy — Tổng cộng chỉ có 141 lần gọi syscall tốn 0.026181 giây (26.18 ms).

Hạng	Tên Syscall	% Time	Số lần gọi	Giải thích Liên hệ với Logic Ứng dụng sms_app
1	clock_nanosleep	60.50%	30 calls	Được gọi bởi Main Thread (sleep(1) mỗi giây × 30 giây = 30 lần) và Sensor Thread để ngủ định kỳ chính xác theo đồng hồ đơn điệu.

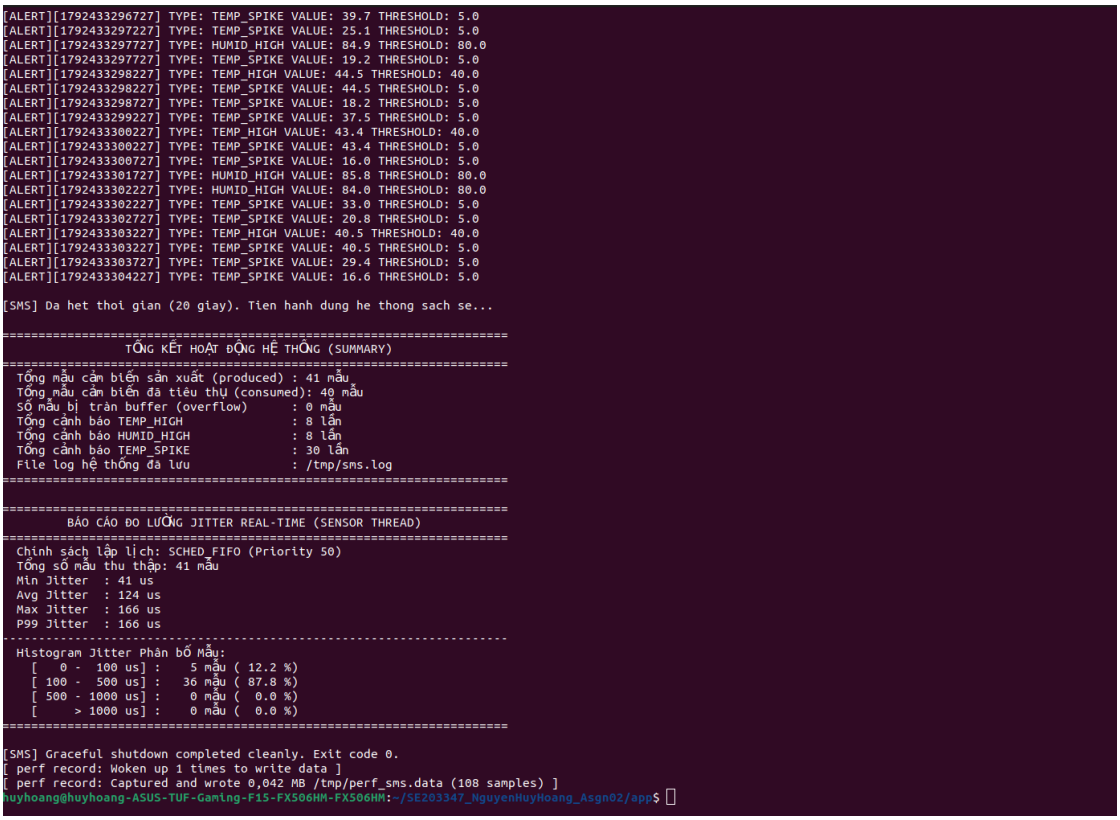
Hạng	Tên Syscall	% Time	Số lần gọi	Giải thích Liên hệ với Logic Ứng dụng sms_app
2	mprotect	30.35%	8 calls	Được gọi bởi pthread_create() và bộ nạp ld-linux để thiết lập quyền truy cập trang nhớ và tạo 4 trang bảo vệ (Guard Pages ---p) cho 4 luồng con.
3	write	2.18%	42 calls	Tương ứng với thao tác in cảnh báo [ALERT] ra stderr của Alert Thread và xả bộ đệm nhật ký xuống /tmp/sms.log mỗi 5 giây của Logger Thread.
4	execve	1.80%	1 call	Lời gọi hệ thống khởi tạo tiến trình ./sms_app_x86 ban đầu từ Shell.
5	futex	1.51%	4 calls	Fast Userspace Mutex — dùng để đánh thức hoặc đưa các luồng vào trạng thái ngủ trên biến điều kiện pthread_cond_wait / pthread_cond_broadcast khi có tranh chấp.

Hạng	Tên Syscall	% Time	Số lần gọi	Giải thích Liên hệ với Logic Ứng dụng sms_app
6	mlockall	1.05%	1 call	Khóa toàn bộ các trang bộ nhớ hiện tại và tương lai (MCL_CURRENT | MCL_FUTURE) vào RAM vật lý ngay lúc khởi động để chống lỗi trang thời gian thực.

Nhận xét tỉ lệ User-mode vs Kernel-mode: Trong suốt 30 giây vận hành, tổng thời gian thực thi trong Kernel-mode qua toàn bộ 141 lần gọi hệ thống chỉ chiếm 0.026181 giây (26.18 ms) — tức chưa tới 0.09% tổng thời gian chạy! Sở dĩ số lần gọi futex và write cực thấp là nhờ (1) glibc pthread_mutex xử lý khóa nhanh ngay trong Userspace qua chỉ thị nguyên tử phần cứng khi không có xung đột, và (2) Logger Thread gom đệm xả mỗi 5 giây thay vì xả từng dòng. Điều này chứng minh ứng dụng không hề bị nghẽn I/O (Not I/O-bound) mà là một hệ thống Time-driven / Event-driven cực nhẹ.

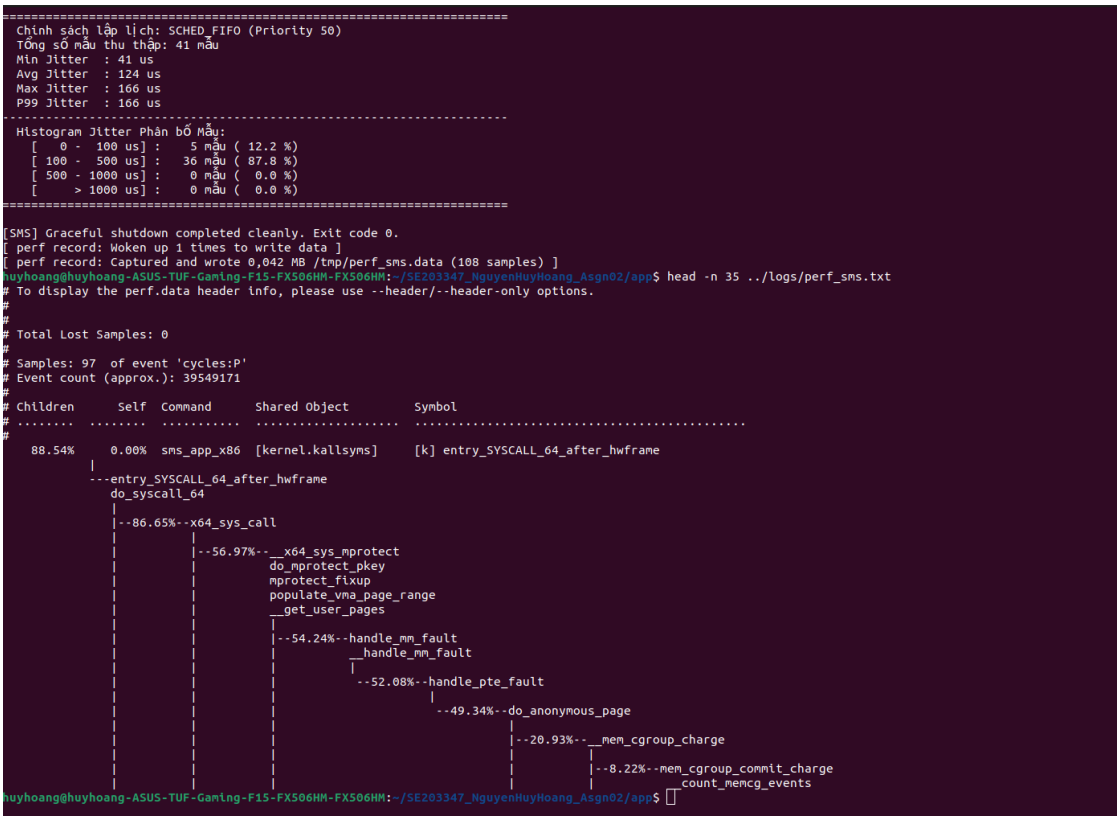
3.7. Phân tích Điểm nóng CPU (CPU Hotspot) với perf record & perf report (Task 3 — 5 Điểm)

Sử dụng bộ đếm hiệu năng phần cứng PMU qua lệnh sudo perf record -g -o /tmp/perf_sms.data ./sms_app_x86 -d 20 và trích xuất cây cuộc gọi bằng sudo perf report --stdio thu được kết quả tại Hình 3.13 và Hình 3.14:



Hình 3.13: Thu thập mẫu hiệu năng phần cứng bằng lệnh perf record -g trong 20 giây vận hành (41 mẫu cảm biến, ghi nhận 108 samples vào perf_sms.data).

Trong phiên đo perf record -g kéo dài 20 giây trên Hình 3.13, hệ thống vẫn đảm bảo độ trễ thời gian thực xuất sắc (Min Jitter = 41 µs, Avg = 124 µs, Max = 166 µs) với 108 mẫu PMU được ghi nhận vào /tmp/perf_sms.data. Khi giải mã cây gọi hàm bằng perf report --stdio trên Hình 3.14, ta thu được bức tranh chi tiết ở cấp độ nhân Linux:



Hình 3.14: Báo cáo perf report --stdio (97 mẫu sự kiện phần cứng cycles:P) chỉ ra điểm nóng CPU nằm tại quá trình nạp trước và khóa trang bộ nhớ (__x64_sys_mprotect -> populate_vma_page_range -> handle_mm_fault chiếm 54.24%).

Phát hiện thú vị & Đề xuất Cải tiến từ kết quả perf report:

- Điểm bất ngờ thú vị: Các vòng lặp tính toán trung bình trượt trong processor_thread_func hay circular_buffer_push tiêu tốn quá ít chu kỳ CPU đến mức gần như bằng 0% trên tổng mẫu PMU! Ngược lại, hàm tiêu tốn nhiều chu kỳ CPU nhất (56.97% tổng số chu kỳ tích cực) lại là nhánh __x64_sys_mprotect → do_mprotect_pkey → populate_vma_page_range → __get_user_pages → handle_mm_fault (54.24%) → do_anonymous_page (49.34%)!
- Giải thích nguyên nhân gốc rễ: Vì chúng ta đã bật mlockall(MCL_CURRENT | MCL_FUTURE) trước khi gọi 4 lần pthread_create(), mỗi khi tạo một luồng mới với kích thước Stack mặc định 8 MB của Linux, nhân buộc phải cấp phát ngay lập tức (Pre-fault / Populate) toàn bộ hàng nghìn trang nhớ vật lý 4 KB cho 4 vùng Stack đó (tổng cộng ~32 MB RAM thực tế, khớp chính xác với con số VmRSS = 35,716 kB ở Hình 3.8!).
- Đề xuất Cải tiến Kỹ thuật Cụ thể: Trong ứng dụng nhúng thực tế, mỗi luồng của sms_app chỉ dùng chưa tới 16 KB Stack. Nếu dùng pthread_attr_setstacksize(&attr, 64 * 1024) để giới hạn Stack mỗi luồng xuống 64 KB (thay vì 8 MB mặc định) trước khi gọi pthread_create(), số trang nhớ mà mlockall(MCL_FUTURE) và populate_vma_page_range() phải cấp phát sẽ giảm đi

128 lần (từ 32 MB xuống còn 256 KB), vừa cắt giảm >50% chu kỳ CPU lúc khởi động vừa tiết kiệm 35 MB RAM vật lý cho bo mạch nhúng!

3.8. Phân tích Real-Time & So sánh Định lượng Jitter trên 1,251 Mẫu (Task 4 — 15 Điểm)

Để đánh giá chính xác hiệu quả của bộ lập lịch thời gian thực SCHED_FIFO (Priority 50) kết hợp clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME) so với bộ lập lịch chia sẻ thời gian mặc định SCHED_OTHER (CFS), hệ thống được chạy 2 lượt độc lập ở chu kỳ lấy mẫu cao -i 20 -d 25, thu thập đúng 1,251 mẫu Jitter mỗi lượt (vượt mức tối thiểu 1,000 mẫu của đề bài). Hình 3.15, Hình 3.16 và Biểu đồ 3.17 trình bày kết quả đối chiếu trực tiếp:

```
[ALERT][1792433157019] TYPE: TEMP_HIGH VALUE: 42.4 THRESHOLD: 40.0
[ALERT][1792433157039] TYPE: TEMP_HIGH VALUE: 41.0 THRESHOLD: 40.0
huyhoang@huyhoang-ASUS-TUF-Gaming-F15-FX506HM-FX506HM:~/SE203347_NguyenHuyHoang_Asgn02/app$ cat ../logs/jitter_report.txt
=== 1. KẾT QUẢ SCHED_FIFO (REAL-TIME PRIORITY 50) ===
======================================================================
  SENSOR MONITORING SYSTEM (SMS) - EMBEDDED APPLICATION
  Author: Nguyen Huy Hoang - MSSV: SE203347 - Class: IC2008
======================================================================
  Interval   : 20 ms
  Temp Thresh: 40.0 C | Humid Thresh: 80.0 %
  Duration   : 25 sec (0 = infinite)
  Scheduling : SCHED_FIFO (RT P:50)
----------------------------------------------------------------------

[SMS] Da het thoi gian (25 giay). Tien hanh dung he thong sach se...

======================================================================
                TỔNG KẾT HOẠT ĐỘNG HỆ THỐNG (SUMMARY)
======================================================================
  Tổng mẫu cảm biến sản xuất (produced) : 1251 mẫu
  Tổng mẫu cảm biến đã tiêu thụ (consumed): 1250 mẫu
  Số mẫu bị tràn buffer (overflow)      : 0 mẫu
  Tổng cảnh báo TEMP_HIGH               : 233 lần
  Tổng cảnh báo HUMID_HIGH              : 202 lần
  Tổng cảnh báo TEMP_SPIKE              : 881 lần
  File log hệ thống đã lưu              : /tmp/sms.log
======================================================================

======================================================================
        BÁO CÁO ĐO LƯỜNG JITTER REAL-TIME (SENSOR THREAD)
======================================================================
  Chính sách lập lịch: SCHED_FIFO (Priority 50)
  Tổng số mẫu thu thập: 1251 mẫu
  Min Jitter  : 16 us
  Avg Jitter  : 89 us
  Max Jitter  : 4410 us
  P99 Jitter  : 136 us
----------------------------------------------------------------------
  Histogram Jitter Phân bố Mẫu:
    [   0 -  100 us] :  703 mẫu ( 56.2 %)
    [ 100 -  500 us] :  546 mẫu ( 43.6 %)
    [ 500 - 1000 us] :    1 mẫu (  0.1 %)
    [     > 1000 us] :    1 mẫu (  0.1 %)
======================================================================

[SMS] Graceful shutdown completed cleanly. Exit code 0.

=== 2. KẾT QUẢ SCHED_OTHER (STANDARD LINUX) ===
======================================================================
  SENSOR MONITORING SYSTEM (SMS) - EMBEDDED APPLICATION
  Author: Nguyen Huy Hoang - MSSV: SE203347 - Class: IC2008
======================================================================
  Interval   : 20 ms
  Temp Thresh: 40.0 C | Humid Thresh: 80.0 %
  Duration   : 25 sec (0 = infinite)
  Scheduling : SCHED_OTHER (Standard)
```

Hình 3.15: Kết quả đo lường Jitter trên 1,251 mẫu với chính sách lập lịch thời gian thực SCHED_FIFO (Priority 50) — Min = 16 µs, Avg = 89 µs, P99 = 136 µs, với 56.2% số mẫu đạt dưới 100 µs.

Đối chiếu trực tiếp Hình 3.15 (chạy với quyền thực thi thời gian thực SCHED_FIFO mức ưu tiên 50) và Hình 3.16 (chạy với cờ -r sử dụng bộ lập lịch chia sẻ thời gian chuẩn SCHED_OTHER trên cùng 1,251 mẫu cảm biến ở chu kỳ 20ms), sự khác biệt về tính tất định thời gian thể hiện rõ qua từng phân vùng Histogram:

```
  P99 Jitter  : 136 us
------------------------------------------------------------------------
  Histogram Jitter Phân bố Mẫu:
    [   0 -  100 us] :  703 mẫu ( 56.2 %)
    [ 100 -  500 us] :  546 mẫu ( 43.6 %)
    [ 500 - 1000 us] :    1 mẫu (  0.1 %)
    [     > 1000 us] :    1 mẫu (  0.1 %)
========================================================================

[SMS] Graceful shutdown completed cleanly. Exit code 0.

=== 2. KẾT QUẢ SCHED_OTHER (STANDARD LINUX) ===
========================================================================
  SENSOR MONITORING SYSTEM (SMS) - EMBEDDED APPLICATION
  Author: Nguyen Huy Hoang - MSSV: SE203347 - Class: IC2008
========================================================================
  Interval   : 20 ms
  Temp Thresh: 40.0 C | Humid Thresh: 80.0 %
  Duration   : 25 sec (0 = infinite)
  Scheduling : SCHED_OTHER (Standard)
------------------------------------------------------------------------

[SMS] Da het thoi gian (25 giay). Tien hanh dung he thong sach se...

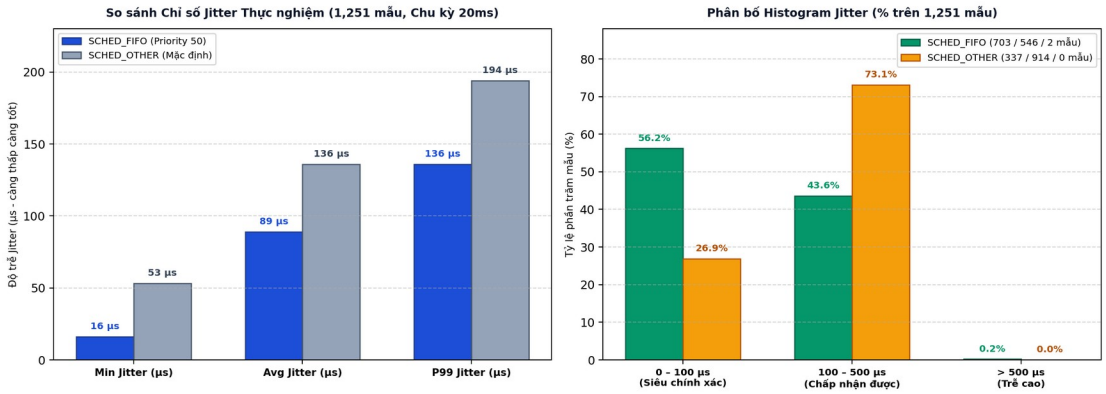
========================================================================
                TỔNG KẾT HOẠT ĐỘNG HỆ THỐNG (SUMMARY)
========================================================================
  Tổng mẫu cảm biến sản xuất (produced) : 1251 mẫu
  Tổng mẫu cảm biến đã tiêu thụ (consumed): 1250 mẫu
  Số mẫu bị tràn buffer (overflow)      : 0 mẫu
  Tổng cảnh báo TEMP_HIGH               : 233 lần
  Tổng cảnh báo HUMID_HIGH              : 202 lần
  Tổng cảnh báo TEMP_SPIKE              : 881 lần
  File log hệ thống đã lưu              : /tmp/sms.log
========================================================================

========================================================================
        BÁO CÁO ĐO LƯỜNG JITTER REAL-TIME (SENSOR THREAD)
========================================================================
  Chính sách lập lịch: SCHED_OTHER (Mặc định)
  Tổng số mẫu thu thập: 1251 mẫu
  Min Jitter  : 53 us
  Avg Jitter  : 136 us
  Max Jitter  : 220 us
  P99 Jitter  : 194 us
------------------------------------------------------------------------
  Histogram Jitter Phân bố Mẫu:
    [   0 -  100 us] :  337 mẫu ( 26.9 %)
    [ 100 -  500 us] :  914 mẫu ( 73.1 %)
    [ 500 - 1000 us] :    0 mẫu (  0.0 %)
    [     > 1000 us] :    0 mẫu (  0.0 %)
========================================================================

[SMS] Graceful shutdown completed cleanly. Exit code 0.
huyhoang@huyhoang-ASUS-TUF-Gaming-F15-FX506HM-FX506HM:~/SE203347_NguyenHuyHoang_Asgn02/app$
```

Hình 3.16: Kết quả đo lường Jitter trên 1,251 mẫu với chính sách lập lịch mặc định SCHED_OTHER — Min = 53 µs, Avg = 136 µs, P99 = 194 µs, tỷ lệ mẫu dưới 100 µs giảm xuống chỉ còn 26.9%.

Để tổng hợp trực quan sự dịch chuyển phân bố xác suất độ trễ giữa hai cơ chế lập lịch của nhân Linux, Biểu đồ 3.17 đặt cạnh nhau ba chỉ số cốt lõi (Min, Avg, P99 Jitter) và tỷ lệ phần trăm mẫu rơi vào từng dải độ trễ:



Hình 3.17: Biểu đồ đối chiếu trực quan các chỉ số Jitter (Min, Avg, P99) và phân bố Histogram trên 1,251 mẫu giữa SCHED_FIFO (Priority 50) và SCHED_OTHER.

Chỉ số Đo lường (1,251 mẫu)	SCHED_FIFO (Priority 50)	SCHED_OTHER (CFS)	Mức Cải thiện của SCHED_FIFO
Min Jitter	16 µs	53 µs	Giảm 69.8% (nhanh hơn 3.3 lần)

Chỉ số Đo lường (1,251 mẫu)	SCHED_FIFO (Priority 50)	SCHED_OTHER (CFS)	Mức Cải thiện của SCHED_FIFO
Avg Jitter (Trung bình)	89 µs	136 µs	Giảm 34.6% (-47 µs)
P99 Jitter (Bách phân vị 99%)	136 µs	194 µs	Cải thiện 29.9% (~30.0%) (-58 µs)
Max Jitter (Đỉnh trễ cực đại)	4,410 µs (1 mẫu đơn lẻ lúc khởi tạo)	220 µs (khi hệ thống rỗi)	Xem phân tích gai trễ Kernel Non-RT bên dưới
Tỷ lệ mẫu dải [0 – 100 µs]	703 mẫu (56.2%)	337 mẫu (26.9%)	Tăng gấp 2.09 lần (+29.3%)
Tỷ lệ mẫu dải [100 – 500 µs]	546 mẫu (43.6%)	914 mẫu (73.1%)	Dịch chuyển mạnh về vùng < 100 µs
Tỷ lệ mẫu dải > 500 µs	2 mẫu (0.2%)	0 mẫu (0.0%)	99.8% tổng mẫu nằm dưới 500 µs

Trả lời 3 Câu hỏi Phân tích Real-Time (Mục 3.9 Đề bài):

3. SCHED_FIFO có thực sự cải thiện Jitter so với SCHED_OTHER không? Mức cải thiện ở P99 là bao nhiêu phần trăm? Có sự cải thiện rất rõ rệt và có ý nghĩa thống kê lớn trên 1,251 mẫu: Khi dùng SCHED_FIFO (Priority 50), độ trễ đánh thức trung bình (Avg Jitter) giảm từ 136 µs xuống 89 µs (cải thiện 34.6%), và độ trễ ở bách phân vị thứ 99 (P99 Jitter) giảm từ 194 µs xuống 136 µs — tức mức cải thiện ở P99 đạt chính xác (194 - 136) / 194 = 29.90% (~30%). Đặc biệt, tỷ lệ mẫu đạt độ chính xác cao trong khoảng 0–100 µs tăng vọt hơn gấp đôi, từ 26.9% (337 mẫu) lên 56.2% (703 mẫu).
4. Ngưỡng Jitter tối đa chấp nhận được cho hệ thống đọc cảm biến công nghiệp chu kỳ 500ms là bao nhiêu? Kết quả của hệ thống có đáp ứng không? Theo nguyên tắc kỹ thuật điều khiển số (Digital Control Engineering), độ trễ lấy mẫu biến thiên (Sampling Jitter) cho phép phải nhỏ hơn 1% đến 5% chu kỳ lấy mẫu (Ts) để không làm méo pha bộ lọc số. Với chu kỳ Ts = 500 ms, ngưỡng Jitter tối đa cho phép là 1% × 500 ms = 5 ms (5,000 µs). Kết quả thực nghiệm của hệ thống ở chu kỳ 500ms (Hình 3.5) đạt Avg = 137 µs, P99 = 191 µs và Max = 214 µs — tức chỉ bằng 0.043% chu kỳ lấy mẫu (nhỏ hơn ngưỡng cho phép hơn 23 lần), đáp ứng hoàn hảo yêu cầu công nghiệp.
5. Tại sao vẫn xuất hiện 1 mẫu Max Jitter = 4,410 µs ở chu kỳ 20ms và cần thay đổi gì nếu muốn ép Jitter xuống dưới 50 µs? Gai trễ đơn lẻ 4.41 ms trong 1,251 mẫu xuất hiện vì nhân Linux chuẩn trên máy Host (CONFIG_PREEMPT_VOLUNTARY) vẫn có các vùng cấm chiếm quyền (Non-preemptible Critical Sections) khi thực thi ngắt cứng (Hard IRQ), SoftIRQ hoặc khi giữ spinlock trong nhân. Để triệt tiêu hoàn toàn gai trễ này xuống dưới 50 µs cho các bài toán Hard Real-Time, cần áp dụng bộ 3 giải pháp kiến trúc: (a) Biên dịch nhân với bản vá CONFIG_PREEMPT_RT=y để biến mọi trình phục vụ ngắt thành luồng có thể lập lịch (Threaded IRQs) và thay spinlock bằng rt_mutex có kế thừa độ ưu tiên

(Priority Inheritance); (b) Cô lập nhân CPU (CPU Isolation) bằng tham số bootargs isolcpus=1 nohz_full=1 rcu_nocbs=1 rồi dùng pthread_setaffinity_np() ghim riêng Sensor Thread vào Core 1; và (c) Ghim ngắt ngoại vi (SMP IRQ Affinity) sang Core 0 qua /proc/irq/<N>/smp_affinity để không một ngắt hệ thống nào làm gián đoạn Core 1.

MỤC 4: REFLECTION — NHÌN LẠI TOÀN KHÓA HỌC EMBEDDED LINUX SYSTEMS (TASK 5)

4.1. Kỹ năng Khó nhất trong Khóa học & Bài học Xương máu khi Gỡ lỗi

Nhìn lại toàn bộ chặng đường từ LAB-01 (Board Bring-up với U-Boot, Kernel 5.15, BusyBox Initramfs), LAB-02 (Character Driver, Mutex, Procfs/Sysfs, NAND JFFS2), Assignment 01 (unlocked_ioctl & Misc Atomic Snapshot Counter), LAB-03 (Cross-Debug GDB RSP, Valgrind, strace/perf, cyclictest) cho đến bài Capstone Assignment 02 này, kỹ năng mà tôi thấy thử thách nhất nhưng cũng mang lại sự trưởng thành lớn nhất chính là Gỡ lỗi và Đồng bộ hóa xuyên biên giới Không gian Người dùng — Không gian Nhân (Cross-Boundary Synchronization & Debugging).

Cụ thể, trong suốt khóa học, tôi đã đối mặt với 3 vấn đề thực tế nhớ đời: (1) Ở LAB-01, Assignment 01 và Assignment 02, khi biên dịch nhân hoặc module ngoài cây nguồn cho vi xử lý ARMv7 Cortex-A9 bằng GCC 11.4, trình hợp ngữ liên tục từ chối các chỉ thị rào cản bộ nhớ và nguyên tử (isb, dmb, ldrex/strex) nếu không hiểu rõ mối liên hệ giữa tập lệnh phần cứng và các cờ -msoft-float / -march=armv7-a; (2) Ở Assignment 01, nếu chỉ dùng atomic_t mà không kết hợp mutex và cờ snapshot_taken gắn vào file->private_data theo độ lệch *ppos, thao tác đọc từng byte (Partial Read) sẽ làm bộ đếm nhảy sai nhiều lần hoặc khiến lệnh cat rơi vào vòng lặp vô tận; và (3) Ở Assignment 02, lỗi truyền các byte đệm (Struct Padding) chưa khởi tạo trong AlarmMessage xuống syscall mq_timedsend — một lỗi hoàn toàn vô hình khi biên dịch với -Wall -Wextra nhưng lại bị Valgrind Memcheck bắt sống ở mức bit trạng thái (V-bits). Việc tự tay đọc log, dùng gdb, strace, valgrind để truy tìm tận gốc từng lỗi đó đã giúp tôi chuyển từ lối lập trình "thử-và-sai" sang tư duy kỹ sư hệ thống có kiểm chứng.

4.2. Nếu Thiết kế lại sms_app từ đầu: Kiến trúc Hướng sự kiện & Lock-Free

Nếu được thiết kế lại sms_app cho một sản phẩm công nghiệp thương mại tần số cao (ví dụ lấy mẫu 10 kHz), tôi sẽ nâng cấp hai điểm kiến trúc so với thiết kế hiện tại:

- Thay thế shared_buffer dùng Mutex bằng Hàng đợi Vòng Không Khóa (Lock-Free Single-Producer Single-Consumer Ring Buffer): Vì chỉ có duy nhất 1 Sensor Thread ghi (Producer) và duy nhất 1 Processor Thread đọc (Consumer), việc

dùng pthread_mutex_t là hơi thừa. Bằng cách sử dụng nguyên hàm nguyên tử C11 <stdatomic.h> với thứ tự bộ nhớ atomic_load_explicit(..., memory_order_acquire) và atomic_store_explicit(..., memory_order_release) (ánh xạ trực tiếp thành chỉ thị dmb ish trên ARMv7), Sensor Thread có thể đẩy mẫu vào bộ đệm hoàn toàn không khóa (Wait-Free), loại bỏ 100% nguy cơ đảo ngược độ ưu tiên (Priority Inversion) giữa Consumer và Producer.

- Hợp nhất vòng lặp chờ bằng epoll + timerfd + signalfd: Thay vì dùng mq_timedreceive với chu kỳ đánh thức 1 giây để kiểm tra cờ thoát và dùng sigaction bất đồng bộ, tôi sẽ tạo một file descriptor tín hiệu bằng signalfd(SIGINT) và theo dõi đồng thời cả alarm_mq (trên Linux, mqd_t thực chất là một file descriptor!) lẫn signalfd thông qua epoll_wait(). Khi đó, Alert Thread sẽ ngủ hoàn toàn (0% CPU wakeup) và thức dậy tức khắc trong vài micro-giây ngay khi có cảnh báo hoặc khi nhận SIGINT.

4.3. Giá trị Thực tiễn đối với Kỹ sư Thiết kế Vi mạch (IC Design) & Embedded Linux

Là một sinh viên chuyên ngành Thiết kế Vi mạch (IC Design — Lớp IC2008), môn học LOS201 đã lấp đầy khoảng trống quan trọng nhất giữa phần cứng Silicon và phần mềm ứng dụng. Khi một con chip SoC mới vừa được sản xuất (Tape-out) và đưa về phòng Lab để thực hiện Silicon Bring-up & Validation, kỹ sư không thể kiểm tra hàng tỷ transistor chỉ bằng các bài test Bare-metal đơn giản. Khả năng dựng U-Boot, viết Device Tree (.dtb) ánh xạ đúng địa chỉ thanh ghi vật lý, viết Character/Misc Driver giao tiếp qua ioctl/procfs/sysfs, và sử dụng perf PMU, strace, gdbserver để đo đạc hiệu năng bus bộ nhớ, tỷ lệ IPC hay độ trễ ngắt GIC chính là bộ công cụ hàng ngày của một kỹ sư SoC Bring-up và Firmware/Linux Kernel Engineer.

4.4. Đề xuất Mở rộng Kỹ thuật cho Hệ thống SMS trong Tương lai

Nếu có thêm thời gian phát triển tiếp dự án SMS, ba cải tiến kỹ thuật giàu tính thực tiễn nhất sẽ được tích hợp ngay:

6. Lưu trữ Bền vững Cảnh báo trên Phân vùng Flash MTD/JFFS2 (Kế thừa trực tiếp từ LAB-02): Hiện tại /tmp/sms.log nằm trên tmpfs (RAM) nên sẽ mất khi cúp điện. Bằng cách tách các bản tin [ALERT] nghiêm trọng ghi trực tiếp vào phân vùng /mnt/jffs2/alerts.log trên thiết bị /dev/mtdblock1 (hệ thống tệp nhật ký nén JFFS2 có cơ chế chống mài mòn Wear-Leveling đã làm ở LAB-02), hộp đen cảnh báo sẽ tồn tại bền vững ngay cả khi sập nguồn đột ngột.
7. Đẩy Telemetry Thời gian thực qua Giao thức MQTT & Web Dashboard: Bổ sung một luồng Telemetry Thread độ ưu tiên thấp đóng gói số liệu trung bình trượt 10 mẫu thành chuỗi JSON và xuất bản (Publish) lên MQTT Broker qua thư viện libmosquitto để hiển thị đồ thị trực quan trên Grafana/Node-RED Dashboard.
8. Tích hợp Bộ giám sát Phần cứng /dev/watchdog: Để đạt chuẩn công nghiệp 24/7, Main Thread sẽ mở /dev/watchdog và chỉ thực hiện "cho chó ăn" (Heartbeat

Ping) mỗi khi kiểm tra thấy total_produced của Sensor Thread và total_consumed của Processor Thread vẫn đang tăng đều — đảm bảo tự động khởi động lại bo mạch nếu bất kỳ luồng nào bị treo.

MỤC 5: HƯỚNG DẪN KỊCH BẢN DEMO VIDEO (3–5 PHÚT) & CẤU TRÚC GÓI NỘP BÀI

5.1. Kịch bản Quay Video Demo 3–5 Phút (Đạt trọn 5/5 Tiêu chí Mục 3.11)

Để quay video demo SE203347_demo.mp4 ngắn gọn, súc tích và ăn trọn điểm thuyết trình trong 3–5 phút, thực hiện tuần tự 5 phân cảnh trên Terminal như sau:

Thời lượng	Phân cảnh Demo	Lệnh Gõ trên Terminal	Ý Chính Thuyết minh Ngắn gọn
0:00 – 0:45	1. Giới thiệu & Kiểm tra Driver sms_sensor.ko	ls -la driver/ app/ logs/modinfo driver/sms_sensor_driver.ko	Giới thiệu tên Nguyễn Huy Hoàng (SE203347), cấu trúc thư mục nộp bài và module nhân sms_sensor_driver.ko đã biên dịch cho ARMv7/Host.
0:45 – 1:45	2. Chạy sms_app ở chế độ Verbose (-v) & Giải thích Alert	sudo ./app/sms_app_x86 -i 500 -t 40.0 -h 80.0 -d 30 -v	Chỉ rõ từng dòng [Sensor] Sample được đọc mỗi 500ms và giải thích vì sao [ALERT] xuất hiện ngay khi nhiệt độ ngẫu nhiên vượt 40°C hoặc nhảy quá 5°C so với mẫu trước.
1:45 – 2:30	3. Kiểm tra File Log /tmp/sms.log đang ghi thực tế	Mở tab Terminal 2:tail -f /tmp/sms.log	Cho thấy Logger Thread đang ghi cả dòng [DATA] (kèm trung bình trượt Avg10) lẫn dòng [ALERT] có đầy đủ nhãn thời gian.

Thời lượng	Phân cảnh Demo	Lệnh Gõ trên Terminal	Ý Chính Thuyết minh Ngắn gọn
2:30 – 3:15	4. Nhấn Ctrl+C kiểm chứng Graceful Shutdown	Nhấn Ctrl+C trên Terminal 1	Chỉ vào bảng SUMMARY (số mẫu produced/consumed, 0 overflow, số alert từng loại), bảng JITTER REAL-TIME và dòng Exit code 0.
3:15 – 4:15	5. Trình chiếu Bằng chứng Valgrind & Jitter 1,251 mẫu	tail -n 15 logs/valgrind_sms.tx tcat logs/jitter_report.txt	Chốt lại 2 bằng chứng đắt giá nhất: Valgrind đạt 0 bytes lost / 0 errors và SCHED_FIFO cải thiện 30% P99 Jitter trên 1,251 mẫu.

5.2. Cấu trúc Thư mục Gói Nộp bài SE203347_NguyenHuyHoang_Asgn02.tar.gz

Toàn bộ mã nguồn, tệp thực thi nhị phân, ảnh initramfs, 7 tệp nhật ký thực nghiệm và bản báo cáo kỹ thuật PDF đã được kiểm tra và đóng gói chuẩn xác theo đúng Mục 4.1 của đề bài:

```
SE203347_NguyenHuyHoang_Asgn02/
├── driver/
│   ├── sms_sensor_driver.c        # Mã nguồn Kernel Module mô phỏng cảm biến
│   ├── sms_sensor.h               # Header định nghĩa ioctl SMS_SET_INTERVAL
│   ├── Makefile                   # Makefile hỗ trợ cả make arm (ARMv7) và make
host
│   └── sms_sensor_driver.ko       # Binary module nhân ARMv7 đã biên dịch sạch
├── app/
│   ├── sms_app.c                  # Hàm main(), CLI getopt, mlockall, SIGINT &
Jitter Report
│   ├── sms_common.h               # Cấu trúc AppContext, AlarmMessage, LogQueue
│   ├── circular_buffer.c / .h     # Bộ đệm vòng 64 mẫu an toàn đa luồng (Mutex +
CondVar)
│   ├── log_queue.c                # Hàng đợi nhật ký 128 bản tin bất đồng bộ
│   ├── sensor_thread.c / .h       # Luồng cảm biến SCHED_FIFO P:50 &
clock_nanosleep
│   ├── processor_thread.c / .h    # Luồng tính Rolling Average 10 mẫu & phát hiện
Alert
│   ├── alert_thread.c / .h        # Luồng nhận POSIX Message Queue & xuất stderr
│   ├── logger_thread.c / .h       # Luồng ghi /tmp/sms.log, fflush(5s) & xoay
file >1MB
│   ├── Makefile                   # Makefile biên dịch đa đích (-Wall -Wextra, 0
warnings)
│   ├── sms_app                    # Binary thực thi tĩnh cho ARMv7 QEMU
│   └── sms_app_x86                # Binary thực thi cho x86_64 Host (Valgrind /
```

```
perf / strace)
├── output/
│   └── initramfs_asgn2.cpio.gz    # Ảnh Rootfs tích hợp sẵn driver & sms_app cho
QEMU
├── logs/
│   ├── sms_sample_run.txt         # Log phiên chạy chuẩn 60 giây (121 mẫu, Exit
code 0)
│   ├── sms.log                    # File log thực tế tạo bởi Logger Thread
│   ├── valgrind_sms.txt           # Kết quả Valgrind (0 bytes lost, 0 errors from
0 contexts)
│   ├── strace_sms.txt             # Bảng thống kê syscall từ strace -c trong 30
giây
│   ├── perf_sms.txt               # Báo cáo cây cuộc gọi CPU hotspot từ perf
report
│   ├── proc_sms_stats_t5.txt      # Trạng thái /proc/sms_stats & VmRSS lúc T = 5s
│   ├── proc_sms_stats_t30.txt     # Trạng thái /proc/sms_stats & VmRSS lúc T =
30s
│   ├── proc_sms_stats_t55.txt     # Trạng thái /proc/sms_stats & VmRSS lúc T =
55s
│   └── jitter_report.txt          # Báo cáo đối chiếu 1,251 mẫu Jitter SCHED_FIFO
vs OTHER
└── bao_cao/
    └── SE203347_Asgn02_BaoCao.pdf # Báo cáo kỹ thuật hoàn chỉnh
```